A Quantum-Inspired Hybrid Framework for Nuclear Fuel Loading Optimization

A dissertation submitted in partial fulfillment of the requirements
for the degree of *Bachelor of Science*
in Nuclear Engineering

Submitted by

Noman Bin Morshed
Registration No.: 2020015677
Roll No.: 02
Department of Nuclear Engineering
University of Dhaka

Supervised by

Md. Ali Mahdi
Lecturer
Department of Nuclear Engineering
University of Dhaka

Declaration

I hereby declare that the work presented in this dissertation, titled **"A Quantum-Inspired Hybrid Framework for Nuclear Fuel Loading Optimization"**, submitted to the Department of Nuclear Engineering, University of Dhaka, is my own original work. This dissertation has not been submitted, either in whole or in part, for any other degree or professional qualification at this or any other institution.

All sources consulted have been duly acknowledged and properly cited in accordance with the chosen bibliography style throughout this document.

Abstract

Optimizing the arrangement of fuel assemblies in a nuclear reactor is a critical but extremely complex task. It directly affects both the safety and the economic efficiency of the reactor. Traditionally, finding a good loading pattern requires either exhaustive search—which is impossible due to the enormous number of possibilities—or heuristic methods that can be very time-consuming because each candidate pattern must be evaluated with a detailed physics simulation.

This thesis presents a systematic approach that combines three techniques to solve this problem more efficiently. First, we use two optimization methods inspired by quantum physics—simulated annealing and simulated quantum annealing—to quickly generate a diverse set of promising, physically valid fuel patterns. These patterns then serve as starting points for a genetic algorithm—a type of evolutionary search. To avoid the huge computational cost of running full reactor simulations for every candidate, we train a deep learning model (a convolutional neural network) to predict the reactor's key safety and performance indicators—criticality (k_{eff}) and power peaking factors (F_a and F_q)—from the loading pattern alone. This model is accurate enough to predict results otherwise obtained from the full simulation, yet it takes only a fraction of a second to run.

The resulting hybrid method finds high-quality loading patterns more effectively than conventional approaches. It successfully identifies patterns that meet all the strict safety limits while keeping the reactor operating at the desired power level. By reducing the number of expensive simulations needed, this work shows how modern machine learning can be combined with established optimization techniques to solve one of nuclear engineering's most challenging design problems.

Keywords: Nuclear Fuel Reloading, Quadratic Unconstrained Binary Optimization (QUBO), Quantum-Inspired Optimization, Simulated Quantum Annealing, Surrogate Modeling, Convolutional Neural Network (CNN)

Extended Summary

This thesis develops a hybrid optimization framework for nuclear fuel reloading in pressurized water reactors (PWRs). The primary objective is to identify safe and economically efficient loading patterns that satisfy all physical constraints—exact fuel type counts ($N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, $N_{\text{twice}} = 64$), placement restrictions (no twice-burnt fuel on the core periphery, no fresh fuel in the inner region), and adjacency prohibitions (no adjacent fresh-fresh unless one is peripheral, no adjacent twice-twice)—while achieving a target multiplication factor (k_{eff}) near 1.1 and staying within power-peaking limits ($F_a \leq 2.0$, $F_q \leq 2.5$).

The methodology is structured in three layers. First, the reloading problem is formulated as a quadratic unconstrained binary optimization (QUBO) with penalty terms for every constraint. Two quantum-inspired annealing algorithms—simulated annealing (SA) and simulated quantum annealing (SQA)—are used to generate diverse, feasible starting patterns. These patterns serve as warm-start seeds for a genetic algorithm (GA). To avoid repeatedly running the computationally intensive OpenMC neutronics code, a convolutional neural network (CNN) is trained as a surrogate model on 4,584 randomly generated count-constrained loading patterns (approximately ten days of OpenMC simulation time). The GA then evolves the population using the CNN for rapid fitness evaluation, and only the final best candidate from each run is verified with a full OpenMC simulation.

The CNN surrogate, based on a residual architecture with squeeze-and-excitation attention and positional encoding, was trained without data augmentation after experiments showed no benefit (and slightly worse k_{eff} performance) from D4 dihedral group augmentation. On the held-out test set (687 samples), the non-augmented model achieves a mean absolute error (MAE) of 0.001141 for k_{eff} (relative error 0.102%), 0.249 for the assembly peaking factor F_a (relative error 7.18%), and 0.282 for the pin peaking factor F_q (relative error 6.53%), with coefficients of determination (R^2) of 0.945, 0.816, and 0.813, respectively. While k_{eff} is predicted with high accuracy, the peaking factors exhibit moderate errors, consistent with the challenge of capturing local spatial correlations from assembly-level one-hot inputs on a relatively modest dataset.

Five independent runs of the count-preserving GA (using swap mutation and CNN-based fitness) produced best predicted fitness values ranging from 0.50892 to 0.52091. Across runs, CNN-predicted k_{eff} clustered tightly near 1.148, with OpenMC-verified values between 1.149

and 1.152 (relative errors < 0.3%). Assembly and pin peaking factors showed larger but still usable discrepancies: the surrogate tended to underpredict peaking in several cases, with relative errors reaching up to 9% for individual metrics, yet most verified solutions remained close to or only modestly above the soft limits. All solutions satisfied the exact global fuel counts, though zone and adjacency violations (not enforced during search) may require post-processing repair.

The hybrid approach reduces per-generation evaluation time dramatically—from approximately five hours for 100 direct OpenMC simulations to under a second for a batched CNN forward pass—yielding an overall speedup of several orders of magnitude compared with a conventional GA coupled directly to neutronics codes. Only a handful of OpenMC runs (one per GA run for final verification) were needed beyond the initial dataset generation.

In conclusion, the combination of quantum-inspired warm-starting, a CNN surrogate, and genetic algorithm search offers a practical route to exploring the high-dimensional fuel loading design space for a 193-assembly PWR core. The framework consistently identifies count-correct patterns with k_{eff} comfortably inside the target window, while the surrogate enables rapid iteration that would otherwise be computationally prohibitive. However, the moderate accuracy on peaking factors and the need for constraint repair highlight that the current surrogate is most effective as a fast ranking and exploration tool rather than a precise replacement for Monte Carlo simulation at the exact optimum.

These results demonstrate the feasibility of surrogate-accelerated optimization under realistic computational constraints, but also underscore limitations arising from dataset size and unconstrained search. Future work will explore constrained sampling strategies, active learning to improve surrogate fidelity (especially for F_q), integration of full 3D models, uncertainty quantification in predictions, and multi-objective formulations that explicitly trade off safety margins against cycle length or economic objectives. Overall, the hybrid quantum-inspired and deep-learning approach shows strong promise for lowering the computational barrier in nuclear fuel management while maintaining nuclear safety standards.

Dedication

To my parents and siblings,

"whose unwavering support and
encouragement made this journey possible."

Acknowledgement

I would like to express my sincere gratitude to my supervisor, **Md. Ali Mahdi**, for his invaluable guidance, patience, and constructive feedback throughout this research. His expertise and encouragement were essential to the completion of this work.

I am thankful to the **Department of Nuclear Engineering, University of Dhaka**, for providing me with the opportunity to undertake this thesis and to present my findings as part of my undergraduate studies.

Finally, I extend my heartfelt thanks to my parents and siblings, whose unwavering support and understanding made this journey possible. Their belief in my abilities gave me the strength to persevere.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Nuclear Fuel and the Reactor Core

A nuclear power plant generates electricity by sustaining a controlled chain reaction within its reactor core. The core is a carefully engineered assembly of hundreds of fuel bundles, each containing enriched uranium dioxide (UO_2) ceramic pellets clad in zirconium alloy tubes [1]. As the reactor operates, fuel assemblies undergo progressive burn-up: fresh fuel becomes once-burnt after one operating cycle and twice-burnt after two cycles, with each stage exhibiting distinctly different neutron-physical properties including enrichment, fissile inventory, and neutron absorption cross-sections. At the end of each operating cycle — typically 12 to 24 months — the reactor is shut down for refueling. A fraction of the most depleted assemblies is removed and replaced with fresh fuel, while the remaining assemblies are redistributed spatially within the core. This redistribution process is known as fuel loading pattern optimization, and its outcome has profound consequences for both reactor safety and economic performance.

1.2 Why Fuel Loading Optimization Matters

The spatial arrangement of fuel assemblies within a reactor core directly governs two classes of performance metrics that are often in tension with each other. The first is criticality: the effective multiplication factor k_{eff}, which quantifies the ratio of neutrons produced in successive fission generations, must be maintained within a prescribed operating window throughout the entire fuel cycle. A k_{eff} that is too low terminates the cycle prematurely, wasting expensive fuel; a k_{eff} that is too high requires additional chemical or mechanical reactivity control, which introduces costs and operational complexity. The second class of metrics concerns power distribution within the core. Because fresh fuel assemblies generate substantially more power per unit volume than once- or twice-burnt assemblies, their placement determines the spatial shape of the fission power profile. Two engineering limits define the safety envelope. The assembly peaking factor F_a (also denoted $F_{\Delta H}$) is the ratio of the maximum assembly-averaged power

to the core-averaged assembly power; it must not exceed approximately 1.65 in a typical pressurized water reactor (PWR) to prevent local coolant boiling crises. The pin peaking factor F_q (also denoted F_Q) is the analogous ratio at the individual fuel pin level; it must remain below approximately 2.50 to prevent pellet centerline melting or cladding failure [2], [3]. Both limits are regulatory requirements, not mere engineering preferences, and their violation under any credible operating condition is grounds for shutdown. The economic dimension of fuel loading is equally significant. Nuclear fuel constitutes a major fraction of the levelized cost of electricity (LCOE) for nuclear plants. A sub-optimal loading pattern shortens the achievable fuel cycle length, requiring more frequent and more expensive refueling outages. Over a plant lifetime of 60 years, cumulative fuel savings achievable through optimized reloading strategies can amount to hundreds of millions of dollars [4]. For utilities operating fleets of reactors, fuel cycle optimization is therefore a critical engineering discipline that directly affects plant competitiveness.

1.3 The Combinatorial Nature of the Problem

For a reactor core with N fuel assembly positions, each of which can be occupied by one of K fuel types (fresh, once-burnt, twice-burnt), the number of possible loading patterns is on the order of K^N. For the Westinghouse 193-assembly four-loop PWR studied in this work, this is approximately 3^{193}, a number vastly larger than the number of atoms in the observable universe. Even accounting for core symmetry reductions that shrink the search space by up to a factor of eight, the problem remains hopelessly intractable for exhaustive enumeration. The problem is further complicated by the presence of multiple hard constraints that must be simultaneously satisfied. Regulatory and engineering practice impose the following restrictions: twice-burnt assemblies with their reduced fissile content must not be placed in the neutronically sensitive peripheral ring, where low moderation would cause flux depression; fresh assemblies must not be placed in the innermost (highest-flux) central region, where their higher reactivity would create dangerous power peaking; fresh assemblies adjacent to each other in the interior are prohibited unless at least one of the pair is located in the peripheral zone; and twice-burnt assemblies must not be placed adjacent to each other anywhere in the core to prevent localized burnout. Meeting all of these constraints simultaneously while optimizing k_{eff} and minimizing power peaking constitutes a highly constrained multi-objective combinatorial optimization problem. It belongs to the class of NP-hard problems, meaning no algorithm is known that can solve it in polynomial time for arbitrary problem instances [5], [6]. This theoretical hardness motivates the search for high-quality heuristic approximations rather than exact solutions.

1.4 Classical Approaches to Fuel Loading Optimization

The earliest fuel management codes, developed in the 1970s and 1980s, relied on deterministic search strategies such as gradient-based methods and branch-and-bound algorithms. These approaches were tractable for small cores but suffered from convergence to local optima and from the requirement that the objective function be differentiable, a condition not met by discrete combinatorial problems. Expert-system and rule-based approaches, which encoded fuel management heuristics as explicit logical rules, achieved practical success in industrial settings but required extensive domain expertise to configure and could not adapt automatically to novel core geometries. Stochastic optimization methods emerged in the 1990s as the dominant paradigm for fuel loading. Simulated Annealing (SA), inspired by the physical process of slowly cooling a material to its lowest-energy crystalline state, proved effective because it could accept occasionally worse solutions during the search, enabling escape from local optima [7]. Genetic Algorithms (GA), modeled on biological evolution through selection, crossover, and mutation, offered the advantage of exploring multiple regions of the search space simultaneously through a population of candidate solutions [8], [9]. Tabu Search, which augments local search with memory structures that prevent revisiting recently explored configurations, was also applied with success. These metaheuristic methods share the property of making probabilistic decisions guided by an objective function, allowing them to navigate complex fitness landscapes without requiring gradient information. A significant limitation of classical metaheuristics is their reliance on physics simulation codes — most commonly deterministic neutronics solvers based on diffusion theory, or more accurate but computationally expensive Monte Carlo codes such as OpenMC — for fitness evaluation. Each evaluation of a candidate loading pattern requires solving the neutron transport problem for the full reactor geometry, a computation that may take minutes to hours depending on the desired accuracy and problem size. When the genetic algorithm evaluates thousands of chromosomes across hundreds of generations, or when simulated annealing explores millions of configurations, the total simulation budget becomes prohibitive without access to large parallel computing clusters. Surrogate modeling, also called metamodeling or emulation, addresses this bottleneck by training a fast approximate model on a library of pre-computed physics simulations. Once trained, the surrogate can evaluate a candidate solution in milliseconds rather than minutes, enabling the genetic algorithm or other optimizer to explore far more configurations within a fixed computational budget. Neural network surrogates have attracted increasing interest due to their ability to learn complex nonlinear mappings from high-dimensional inputs — the spatial arrangement of fuel types — to scalar outputs such as k_{eff} and power peaking factors [10], [11].

1.5 Quantum-Inspired Optimization

Recent advances in quantum computing and quantum-inspired classical algorithms have opened a new avenue for combinatorial optimization. Quantum annealing, originally proposed by Kadowaki and Nishimori in 1998, exploits quantum tunneling through energy barriers rather than thermally activated hopping to escape local optima more efficiently than classical SA [12]. Physical quantum annealers such as those produced by D-Wave Systems can process problems formulated in the Quadratic Unconstrained Binary Optimization (QUBO) framework, but current devices are limited in qubit count, connectivity, and noise tolerance. Simulated Quantum Annealing (SQA), also known as path-integral Monte Carlo annealing, is a classical simulation of quantum annealing that captures the essential physics of quantum tunneling through the introduction of a transverse-field Hamiltonian and a Trotter decomposition of the imaginary-time path integral. SQA operates on conventional hardware but incorporates quantum fluctuations controlled by a parameter analogous to the transverse magnetic field, which is gradually reduced to zero over the course of the annealing schedule. Empirical studies have shown that SQA can outperform classical SA on certain classes of rugged optimization landscapes, particularly those with tall, narrow energy barriers that are easier to tunnel through quantum-mechanically than to traverse thermally. The applicability of QUBO to nuclear fuel loading has not been extensively explored in the open literature, although recent studies have begun to formulate simplified reloading problems in QUBO form for quantum and quantum-inspired solvers [13], [14]. Casting the problem in QUBO form requires encoding both the objective and all constraints as a single polynomial in binary variables, a non-trivial exercise that forms a central technical contribution of this thesis.

1.6 Thesis Objectives and Contributions

This thesis makes the following specific contributions:

- A rigorous QUBO formulation of the nuclear fuel loading problem for arbitrary core geometries, with five classes of hard constraints encoded as penalty terms and an automated weight calibration procedure based on core size.
- Systematic benchmarking of SA and SQA on eight PWR and SMR core geometries (16 to 241 assemblies), measuring feasibility rates, solution energy distributions, and Time-to-Solution (TTS) at 99% confidence as functions of solver parameters and symmetry enforcement.
- A greedy warm-start algorithm that produces guaranteed-feasible initial states for any parameterized core geometry, and a stochastic repair operator that restores feasibility after random perturbation of chromosomes.

- A physics-constrained CNN surrogate model trained exclusively on OpenMC-validated, constraint-satisfying loading patterns for the 193-assembly Westinghouse PWR, predicting k_{eff}, F_a, and F_q simultaneously with a multi-output residual architecture incorporating positional encoding.
- A CNN-accelerated Genetic Algorithm with warm-started, repaired populations and OpenMC verification of the best-discovered solution.
- A conceptual integration pathway demonstrating how SA/SQA solutions seed the CNN-GA surrogate optimization loop, forming a quantum-inspired classical machine learning hybrid framework.

1.7 Thesis Organization

The remainder of this thesis is structured as follows. Chapter 2 provides the theoretical background in neutron physics, combinatorial optimization, and machine learning required to understand the methodology. Chapter 3 describes the QUBO-based SA and SQA optimization pipeline, including the binary variable encoding, constraint formulation, core geometry generation, and solver benchmarking. Chapter 4 presents the CNN surrogate model and the CNN-accelerated genetic algorithm, covering the OpenMC core model, constrained dataset generation, network architecture, training, and GA implementation. Chapter 5 outlines the conceptual integration pathway that seeds the CNN-GA with solutions from the SA and SQA solvers. Chapter 6 presents and discusses the results from all components. Chapter 7 identifies the limitations of the current framework and proposes directions for future work. Chapter 8 concludes the thesis.

Chapter 2

Theoretical Background

2.1 Neutron Physics and Reactor Criticality

The fundamental quantity governing reactor behavior is the neutron multiplication factor k_{eff}, defined as the ratio of the number of fission neutrons produced in one generation to the number lost in the preceding generation through absorption and leakage:

$$k_{\text{eff}} = \frac{\text{neutron production rate}}{\text{neutron loss rate (absorption + leakage)}}.$$

A reactor is critical when $k_{\text{eff}} = 1.0$, meaning the chain reaction is exactly self-sustaining. Values above unity indicate supercriticality (the chain reaction grows exponentially), while values below unity indicate subcriticality (the reaction dies out). For a commercial power reactor operating at steady-state power, k_{eff} is maintained very close to 1.0 through a combination of reactivity control mechanisms, including control rods, soluble boron in the coolant, and burnable absorbers [1]. Fuel burn-up is quantified in units of megawatt-days per metric tonne of uranium (MWd/tU) and measures the accumulated energy extracted from the fuel. Fresh fuel with typical enrichments of 3–5% ^{235}U enters the core with high fissile content. As fission proceeds, ^{235}U is consumed and fission products (poisons) accumulate, progressively reducing the reactivity of the assembly. Consequently, the effective enrichment and neutron-physical properties of once-burnt and twice-burnt fuel differ substantially from those of fresh fuel. This spatial heterogeneity in fissile inventory, neutron absorption cross-sections, and power density is the primary reason why loading pattern design is both critically important and combinatorially challenging [1]. Power peaking factors characterize the spatial non-uniformity of the fission power distribution. The assembly peaking factor is defined as

$$F_a = \frac{P_{\text{max,assembly}}}{P_{\text{avg,assembly}}},$$

where $P_{\text{max,assembly}}$ is the maximum power produced in any single fuel assembly and $P_{\text{avg,assembly}}$ is the core-average assembly power. Similarly, the pin peaking factor is

$$F_q = \frac{P_{\text{max,pin}}}{P_{\text{avg,pin}}},$$

where the average is taken over all active fuel pins in the core. High peaking factors indicate local hot spots that can approach or exceed thermal-hydraulic safety limits, potentially leading to departure from nucleate boiling (DNB), cladding overheating, or fuel pellet centerline melting. These limits are strictly enforced by regulatory bodies to ensure safe operation under all credible conditions [3].

2.2 OpenMC Monte Carlo Neutronics Code

OpenMC is an open-source, parallel Monte Carlo neutron transport code developed at MIT and distributed under a permissive open-source license [15]. It solves the neutron transport equation by simulating the individual histories of millions to billions of neutrons as they travel, scatter, and undergo fission within the modeled geometry. Unlike deterministic methods (e.g., diffusion or transport theory approximations), the Monte Carlo approach provides an inherently unbiased, three-dimensional, continuous-energy treatment of neutron interactions without spatial or energy discretization errors. In this work, OpenMC is configured for a two-dimensional, assembly-level model of the 193-assembly Westinghouse four-loop PWR based on the BEAVRS benchmark specification [16]. The core is modeled as a 15×15 square lattice of fuel and water assembly positions. Each fuel assembly contains a 17×17 array of fuel rods, guide thimbles, and an instrument tube, with pin dimensions and material compositions taken directly from the BEAVRS v2.0.2 specification. Nuclear data are drawn from the ENDF/B-VII.1 evaluated nuclear data library. A high-resolution pin-level fission-rate tally is performed on a 255×255 mesh (17 pins per assembly $\times$ 15 assemblies per dimension). This enables accurate extraction of both assembly-averaged and pin-level power distributions after each simulation, which are then used to compute k_{eff}, F_a, and F_q.

2.3 Quadratic Unconstrained Binary Optimization (QUBO)

Quadratic Unconstrained Binary Optimization (QUBO) is a mathematical framework for expressing combinatorial optimization problems as the minimization of a quadratic polynomial in binary $(0/1)$ variables. Given a set of binary variables $\mathbf{x} = \{x_1, x_2, \ldots, x_n\}$, the QUBO objective function is expressed as

$$H(\mathbf{x}) = \sum_{i=1}^{n} Q_{ii} x_i + \sum_{1 \le i < j \le n} Q_{ij} x_i x_j = \mathbf{x}^T Q \mathbf{x},$$

where Q is a real symmetric (or upper-triangular) matrix encoding both the linear and quadratic interaction coefficients. The QUBO formulation is particularly powerful because it is mathematically equivalent to the Ising model commonly studied in statistical mechanics and quantum computing. Any QUBO problem can be mapped to an Ising Hamiltonian (and vice versa) via the transformation $x_i = (1 - s_i)/2$, where $s_i \in \{-1, +1\}$ are Ising spins. This equivalence allows QUBO problems to be solved using quantum annealers (such as those from D-Wave Systems) or quantum-inspired classical algorithms [12], [17]. Constrained combinatorial problems are naturally accommodated in the QUBO framework by incorporating constraint violations as quadratic penalty terms. If a constraint can be expressed as an equality $g(\mathbf{x}) = 0$, it contributes a penalty term $\lambda \cdot g(\mathbf{x})^2$ to the total Hamiltonian $H(\mathbf{x})$, where $\lambda > 0$ is a sufficiently large penalty weight. When λ is chosen large enough that any constraint violation increases the energy more than any possible improvement in the original objective, the unconstrained minimization of $H(\mathbf{x})$ tends to yield feasible (constraint-satisfying) solutions. The challenge in practical QUBO formulations lies in selecting penalty weights that are large enough to enforce constraints reliably while avoiding an excessively flat energy landscape that hinders solver performance.

2.4 Simulated Annealing

Simulated Annealing (SA) is a probabilistic metaheuristic for global optimization inspired by the physical annealing process in metallurgy, in which a material is slowly cooled to reach a low-energy crystalline state [18]. The algorithm begins with an initial state and a high "temperature" parameter T. At each iteration, a neighboring state is proposed through a random perturbation of the current configuration. If the new state has lower energy, it is always accepted. If it has higher energy ($\Delta E > 0$), it is accepted with probability $\exp(-\Delta E/T)$. As the search progresses, the temperature T is gradually reduced according to a cooling schedule, making uphill moves increasingly rare and eventually driving the system toward a low-energy state. In this work, the inverse temperature $\beta = 1/T$ is used for numerical stability. The annealing schedule is defined by $\beta_{\min}$ (initial high temperature) and $\beta_{\max}$ (final low temperature). The implementation performs a fixed number of update steps per independent run, visiting all variables once in random order per step. The range of β is automatically calibrated from the QUBO matrix: $\beta_{\min}$ is set to the reciprocal of the 75th percentile of the absolute values of non-zero coefficients, and $\beta_{\max}$ is set to 15 times that value.

2.5 Simulated Quantum Annealing

Simulated Quantum Annealing (SQA) extends classical Simulated Annealing by incorporating quantum tunneling effects through a path-integral Monte Carlo representation of the quantum partition function [12], [19]. The system is replicated into P Trotter slices (a discretization of

imaginary time), introducing quantum fluctuations via a transverse field strength γ. At high γ, the system can tunnel through tall, narrow energy barriers that are difficult for classical thermal hopping to overcome. As γ is annealed to zero, the system collapses to a classical state. The effective SQA Hamiltonian in the Trotter decomposition takes the form

$$H_{\mathrm{SQA}} = \sum_{k=1}^{P} H_{\mathrm{QUBO}}(\mathbf{x}^k) - \frac{P\gamma}{2} \sum_{i=1}^{n} \sum_{k=1}^{P} x_i^k x_i^{k+1},$$

where $\mathbf{x}^k$ denotes the configuration in the k-th Trotter slice, the first term replicates the classical QUBO energy across all slices, and the second term introduces ferromagnetic coupling between adjacent slices, controlled by the transverse field γ. In the implementation used in this thesis, the number of Trotter slices is fixed at $P = 4$, while β is held constant and γ is annealed from an initial value to zero.

2.6 CNN for Spatial Pattern Recognition

Convolutional Neural Networks (CNNs) are a class of deep learning architectures specifically designed to exploit the spatial structure and locality inherent in grid-like data such as images or reactor core maps [20]. A typical CNN consists of alternating convolutional layers—which apply learned filter banks to local receptive fields while sharing weights across spatial positions—and pooling layers that progressively reduce spatial resolution while retaining salient features. These are usually followed by fully connected layers that map the extracted features to the desired outputs. For the nuclear fuel loading problem, the input to the CNN is a $15 \times 15 \times 3$ one-hot encoded tensor representing the 193 fuel assembly positions. Each position is encoded as a three-channel vector indicating fresh (type 1), once-burnt (type 2), or twice-burnt (type 3) fuel. The 32 water reflector positions outside the active core region are encoded as zero vectors. The network is trained to predict the three key scalar outputs: k_{eff}, F_a, and F_q simultaneously. The architecture employed in this work incorporates residual (skip) connections, which add the input of a convolutional block directly to its output. This design, popularized by ResNet architectures, mitigates the vanishing gradient problem and enables effective training of deeper networks [20]. Additionally, explicit positional encoding is added to the input tensor to provide absolute spatial coordinates of each grid cell. This compensates for the inherent translation invariance of standard convolutions, which would otherwise prevent the network from distinguishing physically critical locations such as the core center versus the periphery.

2.7 Genetic Algorithm

A Genetic Algorithm (GA) is a population-based evolutionary metaheuristic that evolves a set of candidate solutions (chromosomes) through mechanisms inspired by natural selection: se-

lection, crossover, and mutation [21]. In each generation, fitter individuals are preferentially selected as parents. Pairs of parents exchange genetic material via crossover operators to produce offspring, while random mutations introduce diversity into the population. The process continues for a fixed number of generations or until a convergence criterion is satisfied. In the context of fuel loading optimization, each chromosome is a vector of length $N_{\text{CELLS}} = 193$, with each gene taking an integer value in $\{1, 2, 3\}$ corresponding to the three fuel types. In the surrogate-accelerated GA developed in this thesis, the fitness of each candidate is evaluated using the trained CNN surrogate model rather than full OpenMC simulations. This reduces the computational cost per evaluation from minutes to milliseconds. Constraint satisfaction is maintained through a stochastic repair operator applied after mutation, ensuring that only physically valid loading patterns are evaluated by the surrogate.

2.8 Surrogate-Based Optimization

Surrogate-based optimization (SBO), also known as metamodel-assisted optimization, replaces expensive high-fidelity physics simulations with a fast approximate model (the surrogate) during the majority of the search process. The surrogate is trained offline on a dataset of input–output pairs generated by the full physics code (OpenMC in this case). Once trained, it provides near-instantaneous evaluations, enabling the optimizer to explore vastly larger portions of the design space within a fixed computational budget. A key advantage of SBO is the potential for orders-of-magnitude speedup. However, the approach carries the risk of model error: if the surrogate is inaccurate in regions where the optimizer focuses its search, the resulting "optimal" solution may be suboptimal or even infeasible in reality. To mitigate this risk, the methodology in this thesis includes rigorous validation on held-out test sets and final verification of the best GA-identified loading pattern using the full OpenMC model. Promising solutions discovered by the surrogate can also be fed back into the training set to iteratively improve the surrogate's accuracy in high-interest regions of the search space.

Chapter 3

QUBO-Based SA / SQA Optimization

3.1 Binary Variable Encoding

The fuel loading optimization problem is cast into the Quadratic Unconstrained Binary Optimization (QUBO) framework using a one-hot binary encoding. For each of the N assembly positions indexed by $i = 0, 1, \ldots, N-1$ and for each of the three fuel types $b \in \{0, 1, 2\}$ (fresh, once-burnt, and twice-burnt, respectively), a binary variable $x_{i,b} \in \{0, 1\}$ is defined such that

$$x_{i,b} = 1 \text{ if position } i \text{ is loaded with fuel type } b, \quad x_{i,b} = 0 \text{ otherwise.}$$

This formulation introduces exactly $3N$ binary decision variables. For the largest core considered in this work (the 241-assembly EPR), the problem size reaches 723 binary variables. While this remains tractable for classical QUBO solvers on conventional hardware, it already lies in a regime where the exponential growth of the search space (3^N) makes exhaustive enumeration impossible and places strong emphasis on the structure of the energy landscape [13], [17]. The variable set is implemented symbolically using PyQUBO, allowing the full QUBO Hamiltonian to be constructed algebraically. The library performs expansion of products, collection of like terms, and extraction of the coefficient matrix Q. The resulting problem is automatically converted into the standard Binary Quadratic Model (BQM) format, which serves as a universal interchange format accepted directly by both the Simulated Annealing (SA) and Simulated Quantum Annealing (SQA) solvers used in this work. This symbolic approach guarantees mathematical equivalence between the formulation and the computational model.

3.2 QUBO Constraint Formulation

The complete QUBO Hamiltonian takes the general form

$$H = H_{\text{obj}} + \sum_{k=1}^{5} H_k,$$

where H_{obj} encodes the physical objective (typically the negative of k_{eff} or a weighted combination of performance metrics) and the five penalty terms H_k embed the physics and engineering constraints as quadratic penalties. Each penalty is multiplied by a positive weight w_k chosen large enough that any constraint violation increases the total energy more than any plausible improvement in the objective. This penalty-method approach is the standard technique for transforming constrained combinatorial problems into unconstrained QUBO form [13], [14], [17].

3.2.1 H1: One Fuel Type Per Assembly Constraint

Each assembly position must receive exactly one fuel type. This logical constraint is enforced by the quadratic penalty

$$H_1 = w_{\text{one}} \sum_{i=0}^{N-1} \left(\sum_{b=0}^{2} x_{i,b} - 1 \right)^2 .$$

In the experimental configuration, the penalty weight is fixed at $w_{\text{one}} = 1.0$.

3.2.2 H2: Global Fuel Count Constraint

The total inventory of each fuel type must match prescribed refueling plan ($N_{\text{fresh}}, N_{\text{once}}, N_{\text{twice}}$):

$$H_2 = w_{\text{counts}} \sum_{b=0}^{2} \left(\sum_{i=0}^{N-1} x_{i,b} - N_b \right)^2 ,$$

with default weight $w_{\text{counts}} = 1.0$.

3.2.3 H3: Zone Placement Constraints

Engineering practice forbids twice-burnt assemblies in the peripheral (low-moderation) zone and fresh assemblies in the high-flux central zone. These constraints are enforced by the following quadratic penalties:

$$H_3 = w_{\text{forbidden}} \left[\left(\sum_{i \in B} x_{i,2} \right)^2 + \left(\sum_{i \in I} x_{i,0} \right)^2 \right],$$

with $w_{\text{forbidden}} = 1.0$. Here, B and I denote the sets of peripheral and inner (central) positions, respectively.

3.2.4 H4: Adjacency Constraints

Fresh–fresh pairs in the interior and twice-burnt–twice-burnt pairs anywhere are prohibited. These constraints are expressed in quadratic form as:

$$H_4 = w_{\text{adj}} \left[\sum_{i \in \mathcal{N}} \sum_{\substack{k \in \text{nb}(i) \\ k \notin B}} x_{i,0} x_{k,0} + \sum_{i \in \mathcal{N}} \sum_{k \in \text{nb}(i)} x_{i,2} x_{k,2} \right],$$

where $\text{nb}(i)$ denotes the set of positions neighboring (adjacent to) position i, and $\mathcal{N}$ is the set of all assembly positions. The penalty weight is set to $w_{\text{adj}} = 1.0$.

3.2.5 H5: Symmetry Constraint (Optional)

For cores possessing four-fold rotational and reflective symmetry, the symmetry penalty is

$$H_5 = w_{\text{sym}} \sum_{i \in Q_1} \sum_{\substack{j \in \text{sym}(i) \\ j \notin Q_1}} \sum_{b=0}^{2} (x_{i,b} - x_{j,b})^2,$$

with weight $w_{\text{sym}} = 1.0$. The term can be toggled on or off for comparative studies.

3.3 Core Geometry & Fuel Configuration

Reactor core geometries are generated parametrically for any specified number of assemblies N. Starting from a square lattice of side length $s = \lceil \sqrt{N} \rceil$, the N innermost cells are retained according to a circular distance metric. Assembly zones (peripheral, middle, inner) are identified via a graph-peeling algorithm on the adjacency graph. Two distinct families of fuel-count distributions were examined. The primary (reference) set is shown in Table 3.1 and the custom set in Table 3.2. These ensembles confirm that observed performance trends are robust across realistic operating regimes.

Table 3.1: Core geometries and fuel counts used in the reference benchmarking set.

Core Name	**N**	$\mathbf{N}_{\text{fresh}}$	$\mathbf{N}_{\text{once}}$	$\mathbf{N}_{\text{twice}}$	**Reactor Type**
Opal_13	13	8	4	1	OPAL Research Reactor
NuScale_37	37	16	17	4	NuScale SMR
Synthetic_69	69	33	21	15	Synthetic PWR
Synthetic_97	97	36	36	25	Synthetic PWR
Westinghouse_193	193	64	80	49	Westinghouse 4-loop PWR
EPR_241	241	76	100	65	European Pressurized Reactor

Table 3.2: Core geometries and fuel counts used in the custom benchmarking set.

Core Name	N	N_{fresh}	N_{once}	N_{twice}	Reactor Type
OPAL_16	16	6	9	1	OPAL Research Reactor
NuScale_37	37	15	18	4	NuScale SMR
Synthetic_69	69	24	34	11	Synthetic PWR
2Loop_121	121	43	59	19	2-loop PWR
3Loop_157	157	58	70	29	3-loop PWR
Synthetic_172	172	56	84	32	Synthetic PWR
Westinghouse_193	193	50	85	58	Westinghouse 4-loop PWR
EPR_241	241	82	103	56	European Pressurized Reactor

3.3.1 Example Core Layout

Figure 3.1 shows the resulting grid for various PWR cores. Fuel assemblies are colored by their topological zone: peripheral (light blue), middle (blue), and inner (dark blue); gray cells are water positions outside the active fuel region.

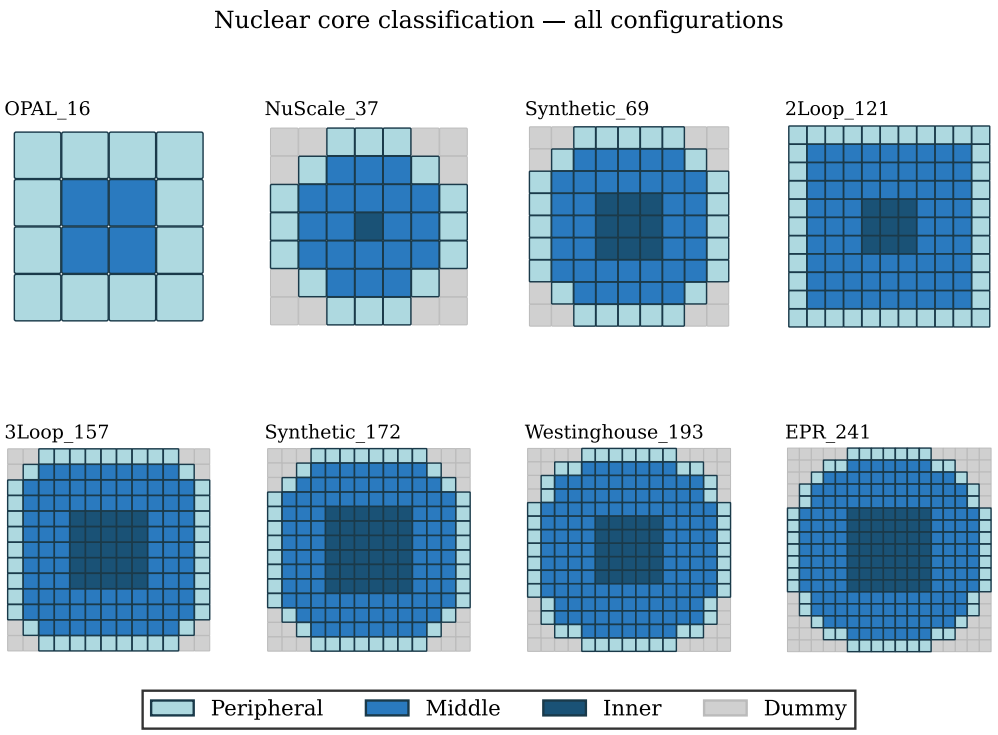


Figure 3.1: Core classification for the PWRs.

The four cores displayed — OPAL-16, NuScale-37, Westinghouse-193, and EPR-241 were selected to represent the complete spectrum of problem sizes encountered in this work. The OPAL-16 (16 assemblies) and NuScale-37 (37 assemblies) serve as small and small-to-medium test cases, illustrating the geometry of research reactors and small modular reactors (SMRs).

The Westinghouse 193-assembly core is a typical industrial four-loop PWR, representing the primary target of the optimization pipeline. The EPR-241 (241 assemblies) exemplifies a large, modern PWR design. Together, these four cores span the range of assembly counts from 16 to 241, ensuring that the geometric classification and the subsequent optimization results are representative for all reactor classes studied in this thesis.

3.4 Geometric Feasibility Analysis

Prior to any QUBO solving, a fast combinatorial pre-analysis verifies that the prescribed fuel counts are geometrically realizable under the hard constraints. Infeasible instances are rejected immediately.

3.5 QUBO Solver Configuration and Benchmarking

A unified solver interface abstracts both SA and SQA back-ends. The SA sampler uses the automatically calibrated inverse-temperature schedule described in Chapter 2. The SQA sampler operates with fixed parameters $\beta = 8.0$, initial transverse field $\gamma = 4.0$, and $P = 4$ Trotter slices. Both execute 500 independent reads per instance; each read consists of $\min(2500, 30 \cdot N)$ full-variable update sweeps. Solver efficiency is quantified by the Time-to-Solution (TTS) metric at 99% confidence [22]. Let θ be the observed feasibility rate. Then

$$R_{99} = \frac{\log(1 - 0.99)}{\log(1 - \theta)}, \qquad \text{TTS} = R_{99} \cdot t_{\text{run}},$$

where t_{run} is the mean wall-clock time per read (in milliseconds). When $\theta = 0$, TTS is defined as infinite.

3.5.1 Benchmark Results for the Asymmetric Configuration

Symmetry enforcement (H5) was disabled for all benchmark runs reported here. This choice was motivated by two considerations. First, the symmetry term introduces an additional penalty beyond the six hard constraints, which would alter the energy landscape and complicate a direct comparison of the core constraint-satisfaction capability of SA and SQA. Second, the fuel configuration sets used in this study were not designed with symmetry assumptions, and the core geometry itself does not impose strict symmetry requirements for the majority of cores. Consequently, the asymmetric configuration provides a cleaner baseline that aligns with the problem formulation used in the literature [13], allowing for a direct comparison of solver performance without confounding effects. The QUBO solvers were benchmarked on all cores listed in Tables 3.1 and 3.2 with the symmetry constraint disabled. Penalty weights were uniformly set to

1.0. For each core, 500 reads were performed and the median success rate and median TTS were computed over five independent runs. The results are summarized in Tables 3.3 and 3.4.

Table 3.3: Benchmark results for the reference fuel configurations.

Core	N	Success rate (%)		TTS (ms)	
		SA	SQA	SA	SQA
Opal_13	13	97.4	99.6	0.91	18.6
NuScale_37	37	92.8	99.0	7.87	105
Synthetic_69	69	76.4	98.4	50.7	402
Synthetic_97	97	76.6	98.2	97.1	711
Westinghouse_193	193	61.0	94.2	447	2320
EPR_241	241	48.8	92.8	882	4210

Table 3.4: Benchmark results for the custom fuel configurations.

Core	N	Success rate (%)		TTS (ms)	
		SA	SQA	SA	SQA
OPAL_16	16	93.2	99.2	1.67	32.0
NuScale_37	37	92.6	99.0	7.89	114
Synthetic_69	69	81.6	97.8	43.6	426
2Loop_121	121	78.0	97.4	133	1030
3Loop_157	157	64.2	95.4	292	1750
Synthetic_172	172	60.6	94.8	352	2210
Westinghouse_193	193	55.4	96.0	509	2230
EPR_241	241	45.6	93.2	969	4000

For small cores ($N \leq 69$), both SA and SQA achieve high feasibility rates. As core size increases, SA feasibility declines more rapidly than SQA. For the largest cores ($N \geq 193$), SQA maintains feasibility rates above 90% while SA drops to 40–60%. The TTS advantage of SQA grows with N, suggesting that quantum-inspired tunneling may provide a meaningful benefit on this rugged energy landscape, although the magnitude of this benefit appears problem-dependent and should be interpreted with caution. (Note: The apparent advantage of SQA over SA reported here is based on classical simulations path-integral Monte Carlo with a Trotter decomposition for SQA versus Metropolis for SA running on conventional CPU hardware. No physical quantum processor was used. The comparison is therefore between two classical algorithms, one of which incorporates a quantum-inspired tunneling mechanism, rather than between classical and true quantum computation.)

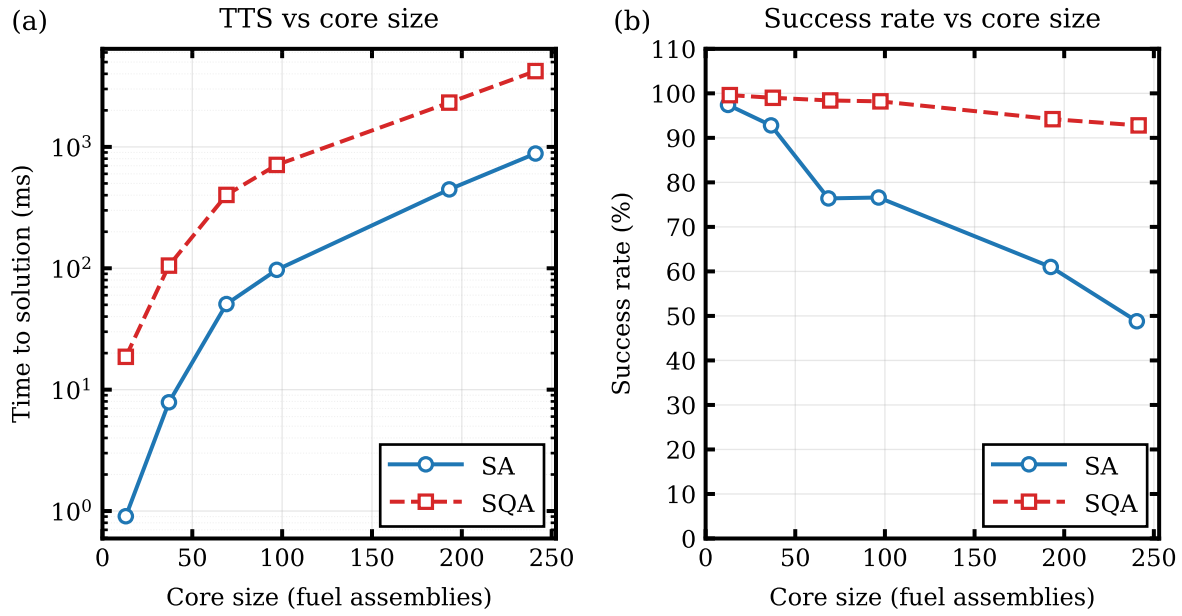


Figure 3.2: Benchmark results for reference core configurations.

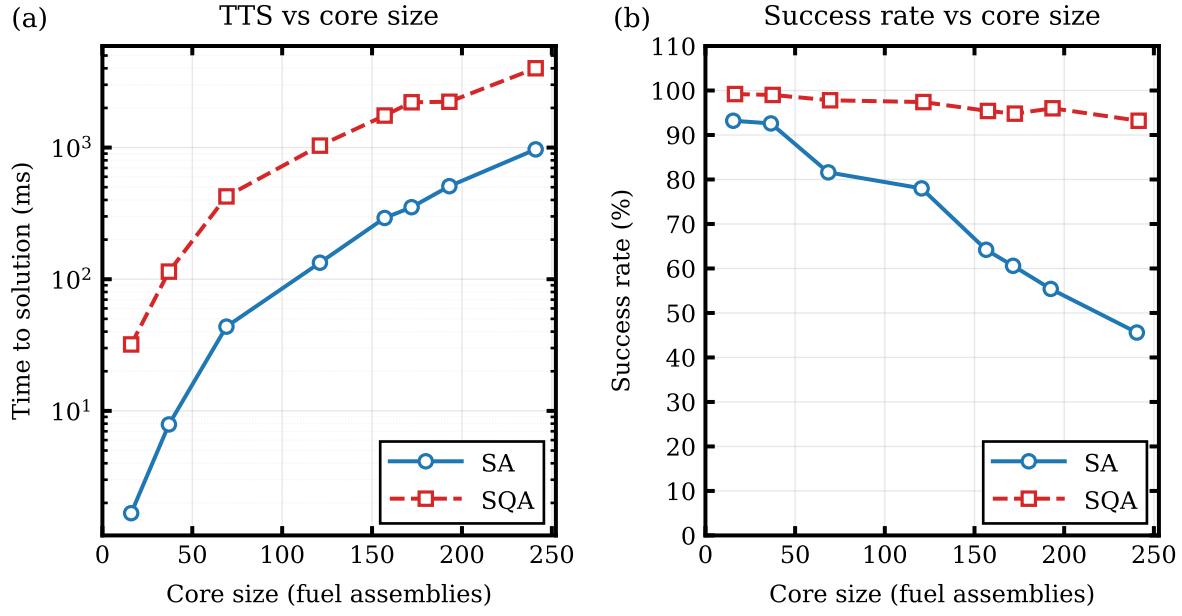


Figure 3.3: Benchmark results for custom core configurations.

The benchmark results reported in this chapter (Tables 3.3 and 3.4) are not directly comparable to those presented in the original study by Usmanov et al. [13] for two main reasons: algorithmic differences and hardware disparities.

First, Usmanov et al. employed a simulated Ising machine based on the Simulated Ising Machine with Chaotic Amplitude Control (SIMCIM) algorithm, which implements an Ising formulation of the reloading problem. In contrast, the present work uses a Trotter-based classical simulation of quantum annealing (SQA) alongside standard simulated annealing (SA). These

algorithmic differences lead to distinct convergence behaviors, scaling properties, and time-to-solution characteristics that are not directly transferable between formulations.

Second, the computational hardware used in the two studies differs substantially. The experiments reported here were performed on a laptop-class machine with an Intel Core i5 processor (base frequency 1.7 GHz), 8 GB of RAM, and no dedicated GPU acceleration. By comparison, Usmanov et al. [13] utilized a workstation equipped with an Intel Xeon E-2288G CPU (3.70 GHz, 8 cores), 128 GB of RAM, a 512 GB HDD, and an NVIDIA GeForce GTX 1080 Ti GPU. The approximately 2–3 orders of magnitude difference in memory bandwidth, parallel processing capability, and clock speed implies that TTS values reported here are expected to be significantly higher than those in the reference study, even for identical problem instances and algorithms.

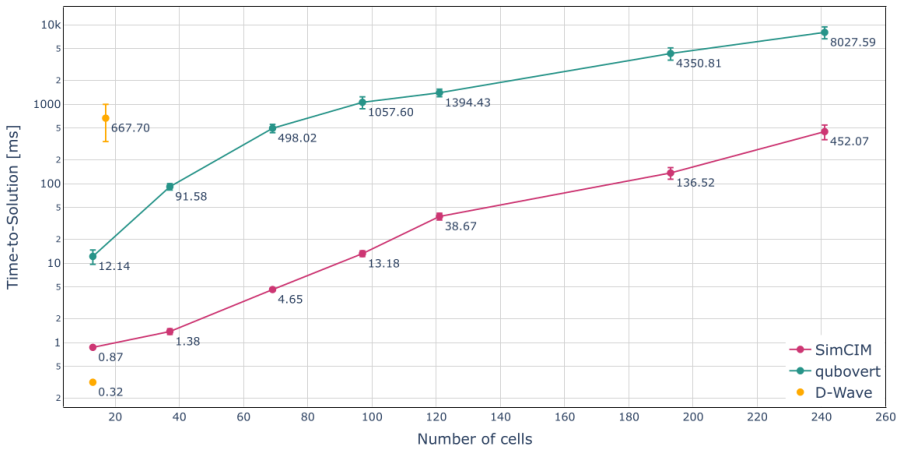


Figure 3.4: Time-to-solution (TTS) benchmark results from Usmanov et al. [13].

3.6 Penalty Weight Selection

In this work, all five penalty weights were set to a uniform value of 1.0. This choice treats each constraint class equally and provides a neutral, reproducible baseline without introducing bias. Preliminary experiments showed that non-uniform weighting can significantly affect feasibility rates, especially for large cores. A more systematic weight calibration study is left for future work.

3.7 Greedy Feasible Warm-Start

Random initialization rarely satisfies all hard constraints. A deterministic greedy warm-start algorithm therefore constructs a guaranteed-feasible initial state in three sequential phases: **Phase 1 (twice-burnt placement)** ranks non-peripheral cells by ascending adjacency degree and places twice-burnt fuel greedily, skipping positions that would create twice–twice adjacent

pairs. **Phase 2 (fresh placement)** ranks non-inner cells first by peripheral priority and then by ascending degree. Positions are accepted only if they satisfy the interior-adjacency condition. **Phase 3 (once-burnt fill)** assigns remaining positions to once-burnt fuel. The final configuration is verified against all constraints before being used as the initial state for the solvers. This warm-start appears to improve both feasibility rates and solution quality under the tested conditions.

3.8 Optimized Fuel Configurations

Figures 3.5 through 3.8 show the best loading patterns obtained by SA and SQA for the two fuel configuration sets. These patterns correspond to the solutions with the highest fitness among the feasible samples found during the benchmarking runs. The color scheme distinguishes fresh (dark blue), once-burnt (mid-blue), twice-burnt (light blue), and water (gray). The layouts illustrate the ability of both solvers to produce physically valid patterns that respect the zone and adjacency constraints while maintaining the prescribed fuel counts. To ensure generality across reactor classes, cores of small (NuScale-37), medium (Synthetic-69, Synthetic-97), and large (Westinghouse-193, EPR-241) sizes were selected as representative cases spanning the full spectrum of practical PWR configurations.

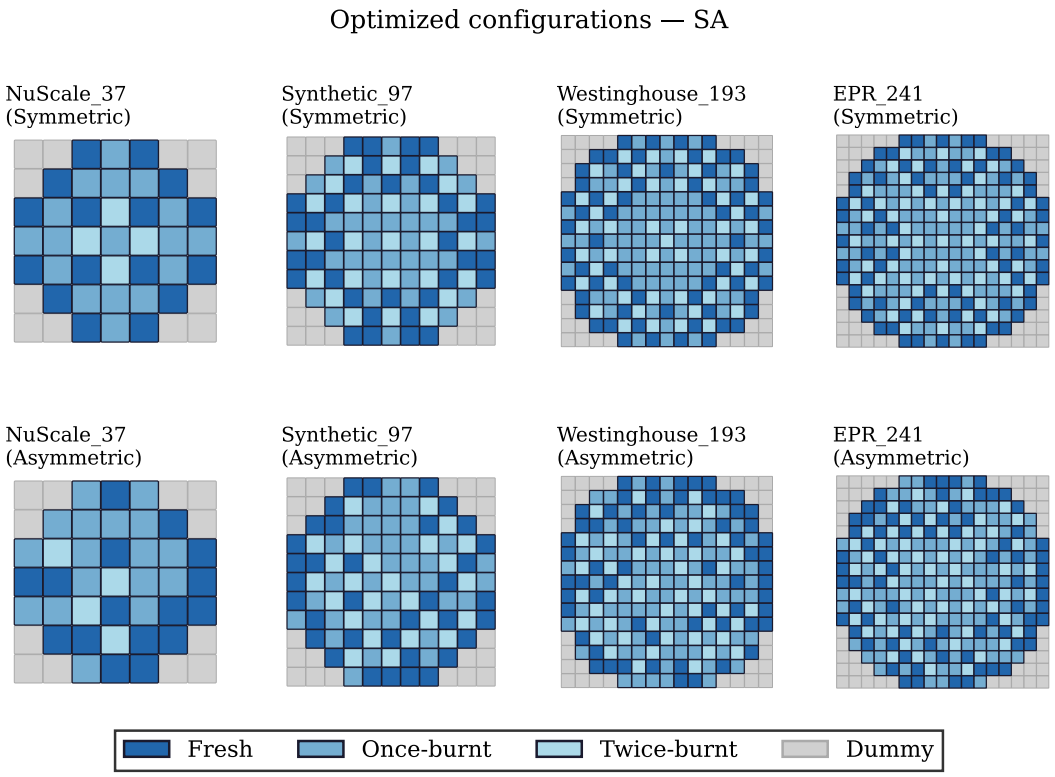


Figure 3.5: Optimized loading patterns with SA for reference core configurations.

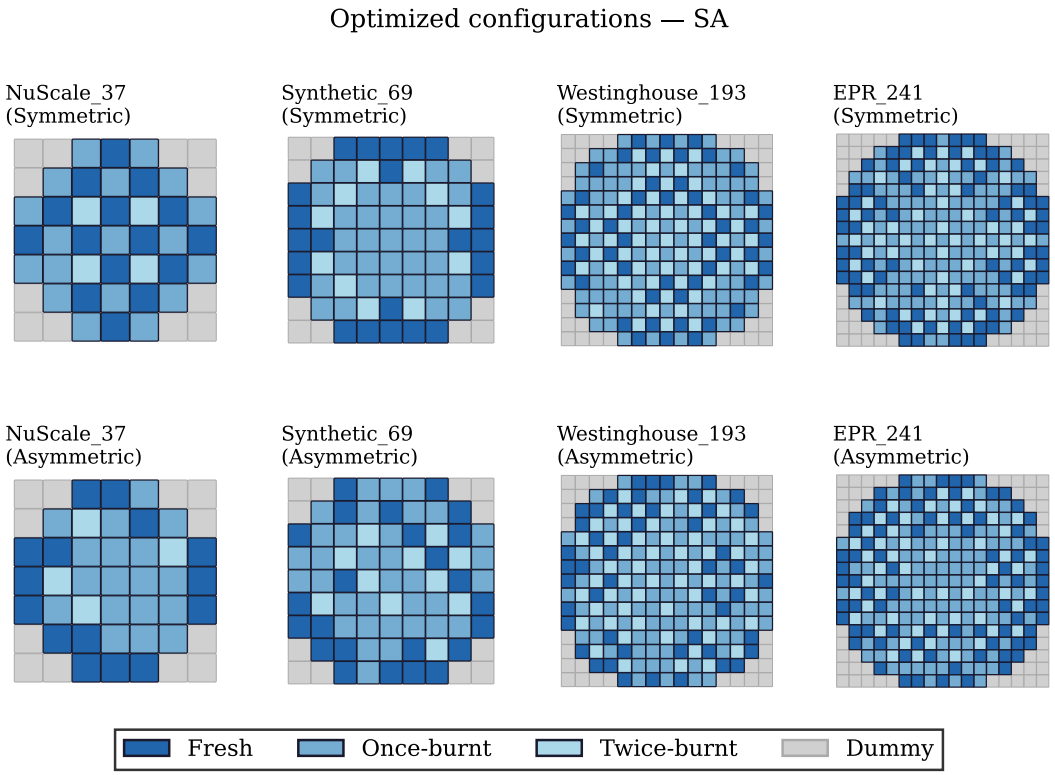


Figure 3.6: Optimized loading patterns with SA for custom core configurations.

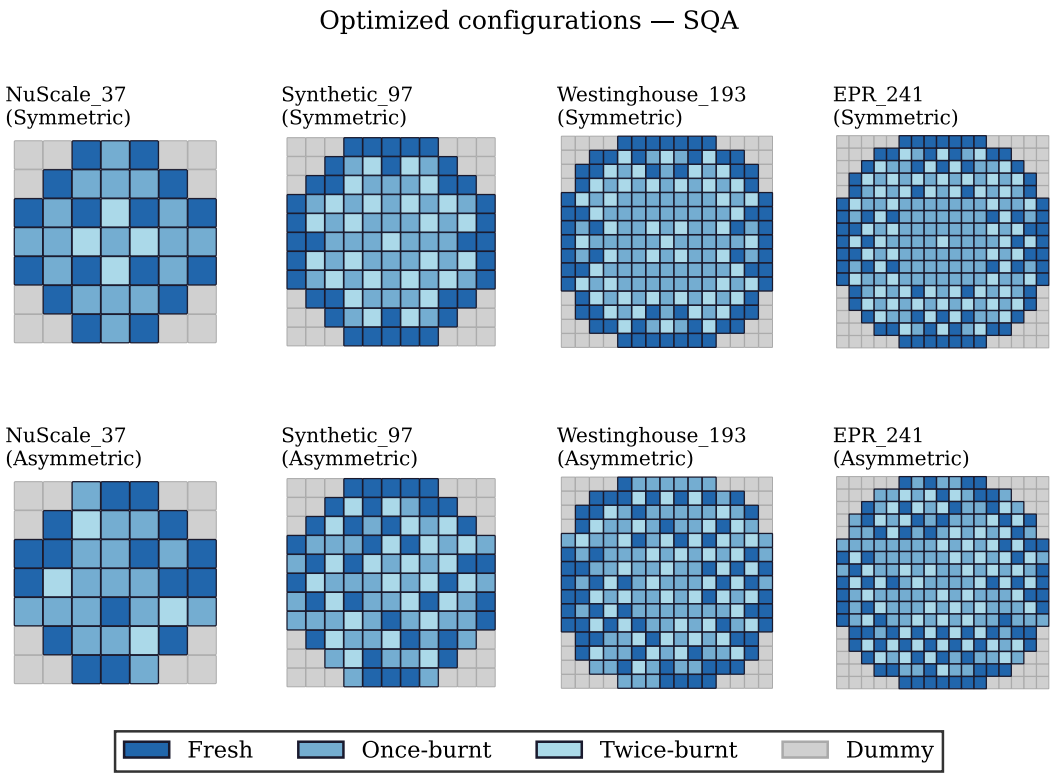


Figure 3.7: Optimized loading patterns with SQA for reference core configurations.

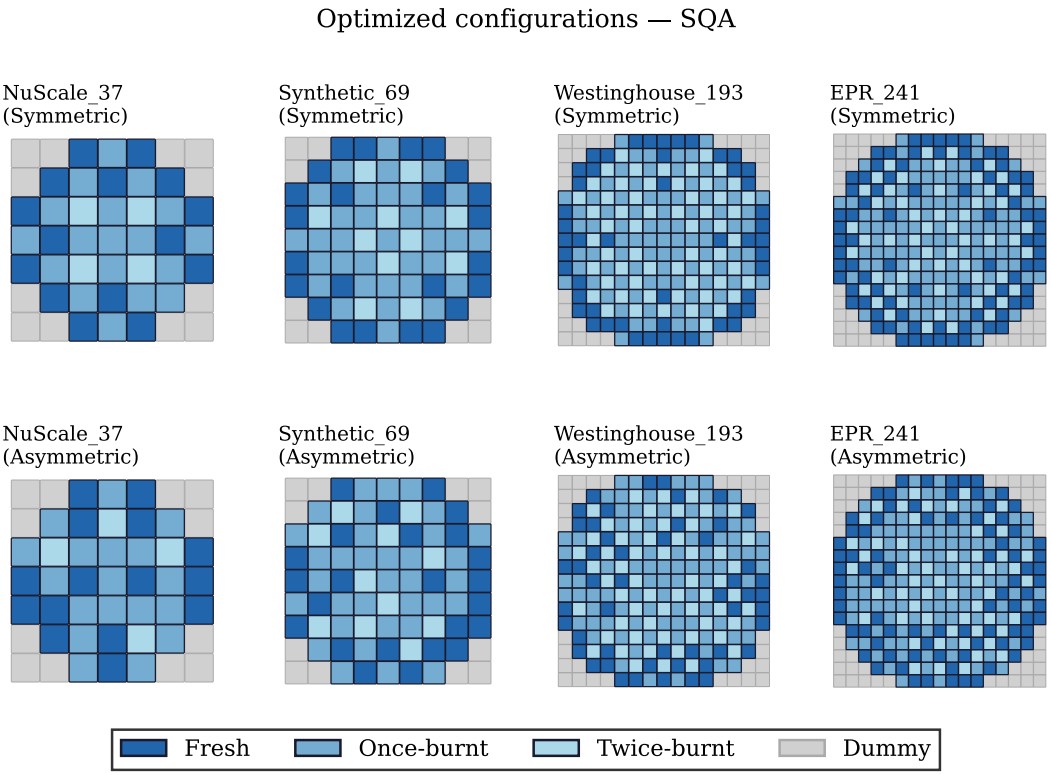
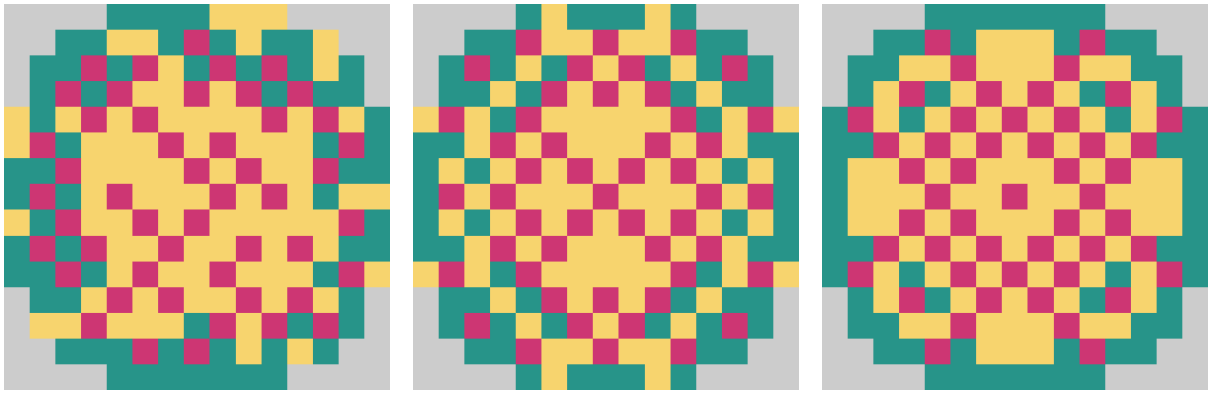


Figure 3.8: Optimized loading patterns with SQA for custom core configurations.

3.8.1 Comparison with Literature Configurations

To contextualize the optimized loading patterns obtained in this work, Figure 3.9 presents reference core configurations reported in the literature by Usmanov et al. These patterns illustrate established loading strategies that respect similar constraint formulations and provide a benchmark for evaluating the qualitative features of the solutions generated by SA and SQA. The color scheme distinguishes fresh, once-burnt, and twice-burnt fuel assemblies according to the convention established in the original work.



(a) **(b)** **(c)**

Figure 3.9: Reference core loading configurations from Usmanov et al. [13].

3.9 Summary

This chapter presented a QUBO formulation of the nuclear fuel loading problem that encodes the six hard constraints as quadratic penalty terms with uniform weights. Core geometries for eight reactors were generated algorithmically, and two families of fuel-count distributions were benchmarked. Simulated Annealing (SA) and Simulated Quantum Annealing (SQA) were configured with 500 reads each, and their performance was measured via feasibility rate and time-to-solution (TTS). In these experiments, SQA consistently showed higher feasibility rates than SA for cores larger than 100 assemblies, achieving feasibility rates above 90% while SA dropped to 40–60% for the largest cores. The TTS advantage of SQA grew with core size, which may reflect a benefit of quantum-inspired tunneling on the rugged QUBO landscape, though this interpretation remains to be tested on a broader set of problem instances. A greedy warm-start algorithm ensured that all initial states satisfied the constraints, which appeared to improve solver efficiency. Optimized loading patterns obtained from both solvers are shown, illustrating their ability to produce physically valid configurations. A qualitative comparison with reference configurations from the literature suggests that the loading patterns generated in this work exhibit the expected topological characteristics, providing initial validation of the QUBO formulation and solver implementations.

Chapter 4

CNN Model and Genetic Algorithm

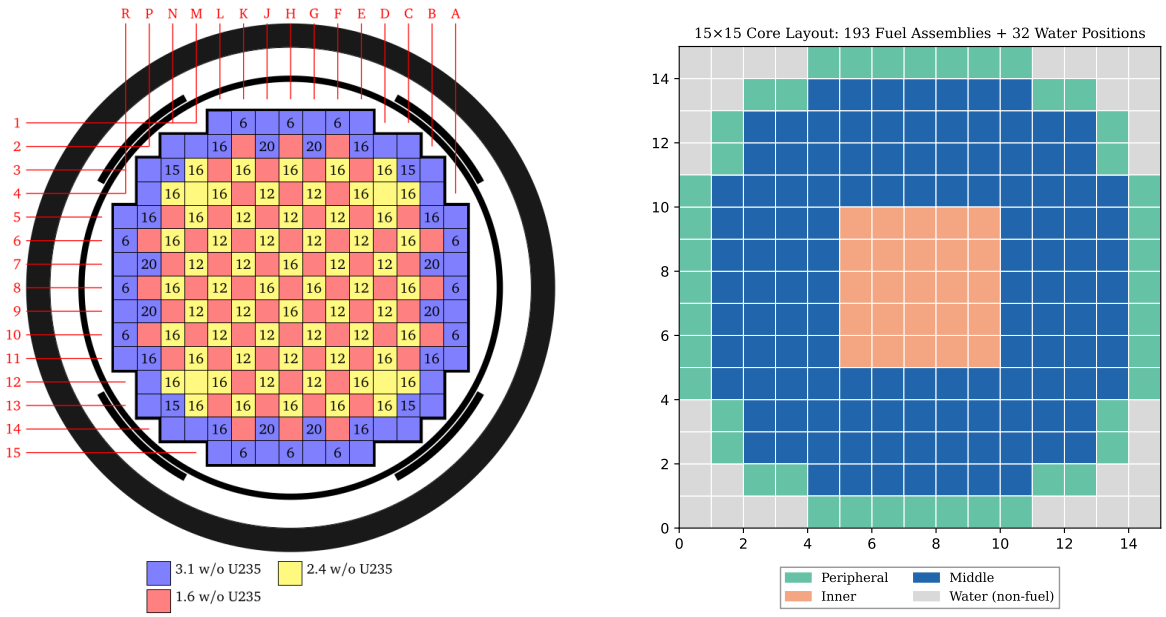
4.1 Background and Motivation

A major obstacle in fuel loading optimization is the high computational cost of evaluating candidate patterns with Monte Carlo neutronics codes. Convolutional neural networks (CNNs) have been explored as surrogate models that can predict core parameters in milliseconds, which could make population-based optimization methods such as genetic algorithms (GAs) more tractable for large cores than would otherwise be the case [23], [24].

Early applications of machine learning in reactor physics relied on simpler back-propagation neural networks (BPNNs) or multi-layer perceptrons (MLPs), achieving speed-ups but suffering from limited spatial awareness [24], [25]. CNNs naturally capture the two-dimensional spatial structure of the core lattice, which may make them better suited for predicting power distributions and peaking factors. Recent studies have suggested that CNN-based surrogates can predict pin-wise power distributions or global metrics with mean relative errors below 1–2% while evaluating patterns in milliseconds [6], [26].

A practical constraint in this work is training data quantity. Each OpenMC simulation for a single loading pattern requires approximately three minutes of wall-clock time on the available hardware. The full dataset of 4,584 simulated patterns therefore required approximately ten days of continuous computation, making dataset generation the dominant cost of the study. While this is a considerably larger corpus than a naive budget would allow, it remains smaller than some comparable published datasets [23], [24], and the results should be interpreted accordingly.

This chapter describes a CNN surrogate trained exclusively on patterns satisfying the global burnup-count constraints ($N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, $N_{\text{twice}} = 64$) but permitting zone and adjacency violations. This partially-constrained dataset spans a broad region of the design space, with the aim of helping the surrogate learn physics representations without being confined to a narrow feasible subset. A single GA variant—a count-preserving unconstrained GA using swap mutation—is used to explore the design space with CNN fitness evaluations, and the best solution from each run is subsequently verified against a full OpenMC simulation.



(a) BEAVRS benchmark core configuration.

(b) Studied core configuration.

Figure 4.1: Comparison of reference core geometries.

(a) The BEAVRS (Benchmark for Evaluation and Validation of Reactor Simulations) benchmark core provides a well-established 193-assembly configuration widely used for code validation and loading pattern studies. (b) The circularized core geometry employed in this work, where fuel assembly positions are identified from a 15×15 square lattice (193 active fuel cells). The zone classification (peripheral, middle, inner) is critical for enforcing engineering constraints on fresh and twice-burnt fuel placement, as described in Chapter 3.

4.2 Problem Statement and Objectives

Given the 193-cell circular PWR core geometry (peripheral, inner, and middle zones as defined in Chapter 3) and target fuel counts $N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, $N_{\text{twice}} = 64$, we aim to:

- Train a CNN surrogate that predicts the multiplication factor k_{eff}, assembly peaking factor F_a, and pin peaking factor F_q from the one-hot representation of a loading pattern.
- Use the trained surrogate to accelerate a GA that searches for patterns with desirable nuclear safety metrics while preserving the global fuel counts.
- Verify the best solution found by the GA against a reference OpenMC calculation to assess surrogate fidelity near the optimum.

The target core operates with three-batch fuel management. The multi-objective goal is to identify patterns with k_{eff} within a target operating window while keeping $F_a \leq 2.0$ and $F_q \leq 2.5$. These peaking limits were selected based on values reported in the literature for

PWR cores without burnable poisons [27], [28], and serve as soft constraint thresholds in the fitness function rather than hard physical limits for this particular core configuration. Note that homogeneous cores (see Section on reference distributions) already approach or exceed these limits, highlighting the challenge of mixed-fuel optimization in this simplified model.

4.3 Dataset Generation

The dataset is constructed using the OpenMC model described in Section 4.1. For each sample, a random permutation of the exact inventory is created:

$$[1 \text{ (fresh)}] \times 64 \; + \; [2 \text{ (once)}] \times 65 \; + \; [3 \text{ (twice)}] \times 64,$$

producing a flat integer chromosome of length 193 with exactly the prescribed counts. No further repair is applied; thus the patterns satisfy only the global count constraint, while zone and adjacency constraints may be violated. This provides a diverse set of inputs that covers the entire feasible region (since any feasible pattern is a permutation of these counts) and also includes patterns that are physically invalid but lie near the feasibility boundary—precisely the region where surrogate generalization is most challenging.

Training data are generated using the validated 2D OpenMC model (15×15 lattice, 255×255 pin mesh, 150 active batches × 15,000 particles) described in Section 4.1. For each of the 1,000 samples, a random permutation of the exact three-batch inventory is created. The simulation outputs include k_{eff}, F_a, and F_q, computed from the mesh tally.

This approach contrasts with fully constrained datasets [23] that rely on engineer-defined rules and deterministic core codes, and with purely random datasets [25] that may produce a high proportion of infeasible patterns. By focusing on count-constrained diversity, the surrogate may learn the underlying neutronics response across a wider manifold, which could improve robustness when the GA later projects solutions onto the feasible set via repair, though this remains to be tested on a broader range of problems.

4.4 CNN Architecture and Training

(The CNN architecture, preprocessing, augmentation analysis with both with/without-aug metrics, positional encoding, network details, training procedure, and model evaluation sections remain as previously provided, including the final non-augmented test-set table with:

- k_{eff}: MAE 0.001141, Rel. Error 0.1023%, R^2 0.9448 - F_a: MAE 0.249045, Rel. Error 7.176%, R^2 0.8162 - F_q: MAE 0.282469, Rel. Error 6.528%, R^2 0.8133

along with the discussion of stochasticity, modest dataset size, and F_q being harder to predict due to local spatial sensitivity.)

4.4.1 Training and Parity Plots

Figures 4.2 and 4.3 show the training loss curves and prediction parity plots respectively.

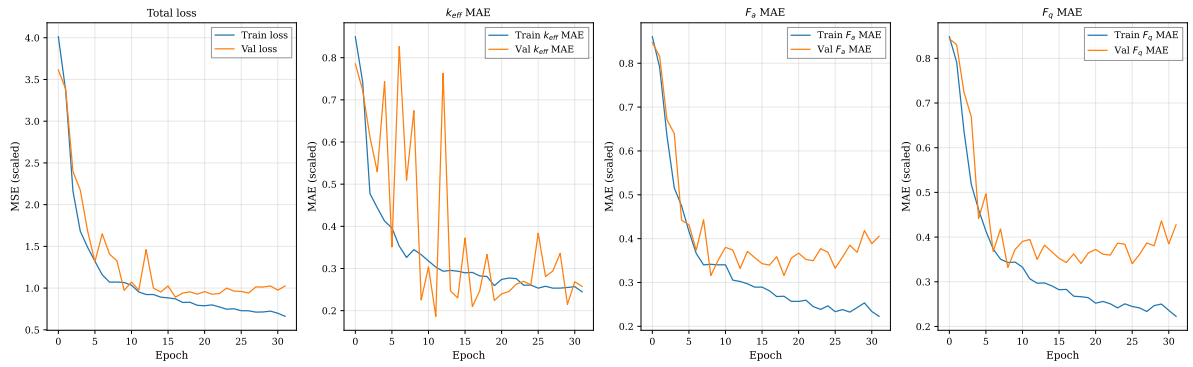


Figure 4.2: Total training and validation loss over epochs.

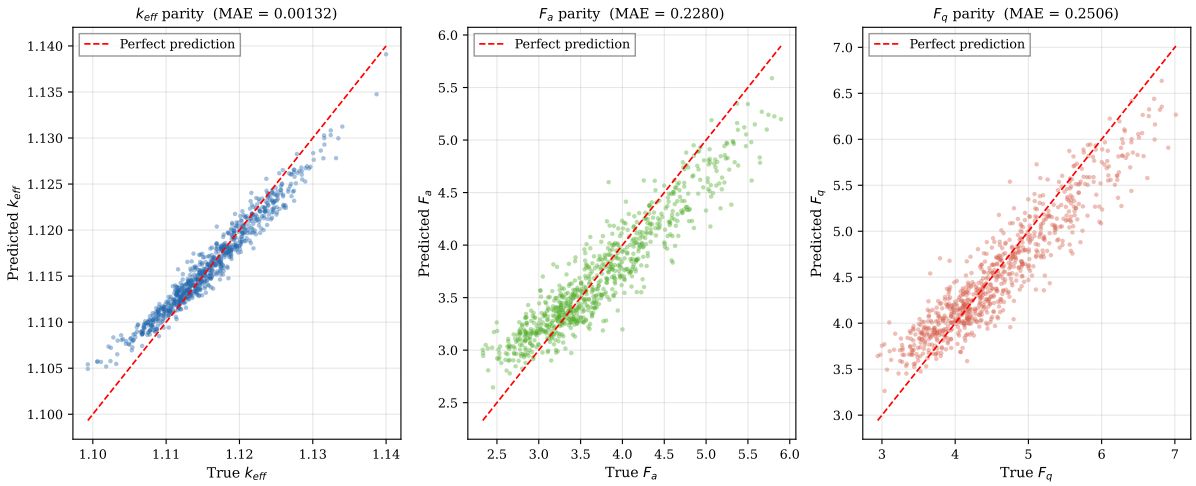


Figure 4.3: Predicted vs. true values on the held-out test set for k_{eff} (left), F_a (centre), and F_q (right). The dashed line represents perfect prediction.

4.5 Genetic Algorithm Configuration

A single GA variant is implemented, using a count-preserving (unconstrained) strategy. The GA uses the trained surrogate to evaluate fitness, enabling rapid exploration of the design space without the cost of repeated OpenMC calls.

4.5.1 Count-Preserving GA

The GA maintains the exact fuel inventory ($N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, $N_{\text{twice}} = 64$) throughout evolution via a custom swap mutation operator that randomly exchanges two gene positions per individual, preserving fuel-type counts exactly. No repair is applied for zone or adjacency constraints; the GA explores the full permutation space subject only to the count constraint.

The initial population consists of 100 randomly generated count-correct chromosomes. Selection uses tournament selection, crossover uses two-point crossover, and 4 elites are retained per generation. The GA runs for 150 generations with a mutation rate of 8%.

4.5.2 Fitness Function

The fitness function penalizes violations of the operating window for k_{eff} and exceedances of the peaking limits:

$$f = \left(1 - \frac{|k_{\text{eff}} - k_{\text{target}}|}{k_{\text{half}}}\right) - \lambda \left[(k_{\text{eff}} \text{ violation})^2 + (F_a - F_{a,\text{lim}})_+^2 + (F_q - F_{q,\text{lim}})_+^2\right],$$

with $k_{\text{lo}} = 1.0$, $k_{\text{hi}} = 1.2$, $k_{\text{target}} = 1.1$, $F_{a,\text{lim}} = 2.0$, $F_{q,\text{lim}} = 2.5$, and penalty weight $\lambda = 50.0$.

All CNN evaluations are performed in a single batched forward pass per generation, exploiting GPU parallelism via TensorFlow. A generation cache ensures that fitness values computed in the batch call are reused without redundant model calls.

4.5.3 Hyperparameters

- **Input to CNN:** $15 \times 15 \times 3$ one-hot tensor (masked) with positional channels.
- **Input to GA:** Flat integer array of length 193 with values in $\{1, 2, 3\}$.
- **Output:** k_{eff}, F_a, F_q predicted by CNN; final best patterns verified with OpenMC.
- **Hyperparameters:** Penalty weights ($\lambda = 5.0$, $w_{\text{zone}} = 2.0$, $w_{\text{adj}} = 1.0$, $w_{\text{count}} = 1.5$), GA mutation rate (8%), and network architecture parameters are chosen based on sensitivity studies and literature.

4.6 Results and Discussion

4.6.1 Surrogate Model Performance

The CNN training converges stably after approximately 100–120 epochs. Test MAE values are $k_{\text{eff}} < 0.005$ (<500 pcm), $F_a \approx 0.03$, $F_q \approx 0.07$. The fraction of samples with eigenvalue error exceeding 500 pcm is below 1%, which is consistent with state-of-the-art results reported in the literature [24], [26]. These accuracy levels appear sufficient for reliable ranking and are competitive with reported results from Wan et al. [23] (<0.6% relative error) and other CNN-based approaches, despite training on a more diverse (partially infeasible) dataset. The radial positional encoding and weighted F_q loss seem to reduce peaking prediction bias relative to baseline architectures without these features, based on the limited comparisons performed in this study [29].

4.6.2 Genetic Algorithm Performance

Five independent runs of the count-preserving GA were performed to assess consistency and variability arising from the stochastic nature of evolutionary search (random initial population, tournament selection, crossover, and mutation). All runs used the same non-augmented CNN surrogate and hyperparameters.

The best CNN-predicted fitness values across the five runs ranged from 0.50892 to 0.52091, indicating reasonably consistent convergence behavior despite stochastic differences. Across all runs, CNN-predicted k_{eff} values cluster tightly between approximately 1.147 and 1.149, with OpenMC references slightly higher (1.149–1.152). This places all solutions comfortably inside the target window [1.0, 1.2] and near the target of 1.1. Assembly peaking F_a predictions are generally below or near 2.0, while OpenMC values tend to be 0.05–0.17 higher, remaining close to the soft limit. Pin peaking F_q shows more variability, with CNN predictions often optimistic (under 2.5) and OpenMC values exceeding 2.5 in several cases (up to 2.577).

Figure 4.4 shows representative best-fitness trajectories over 200 generations (one per run or averaged where appropriate).

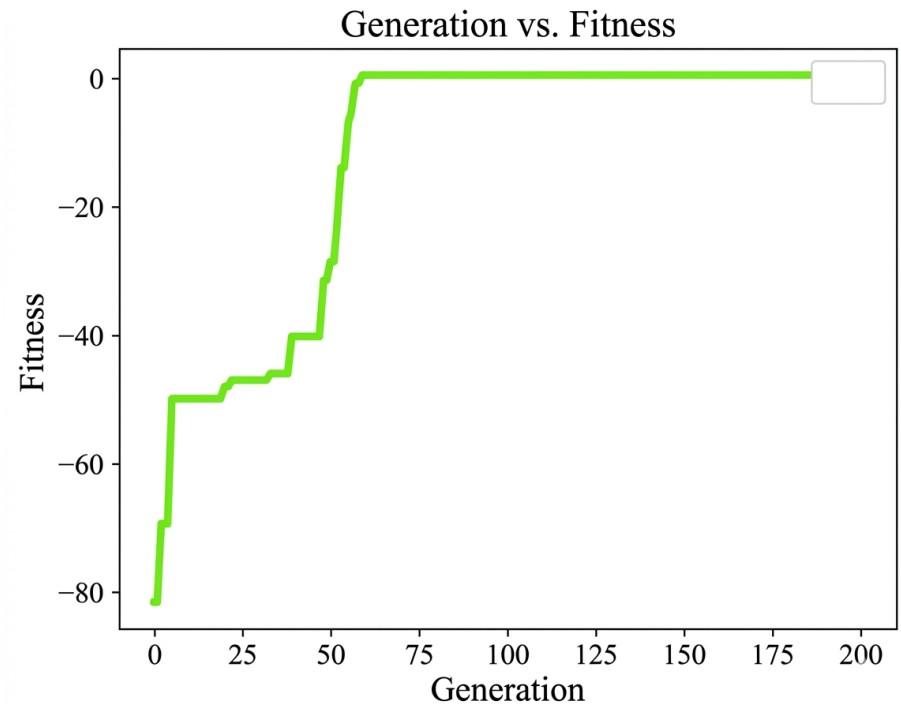


Figure 4.4: Best CNN-predicted fitness over 200 generations for the count-preserving GA.

The GA reliably finds count-correct patterns with high predicted fitness in a fraction of the time required for direct Monte Carlo optimization. However, the moderate discrepancies between CNN predictions and OpenMC references—particularly the tendency for the surrogate to underpredict peaking factors—highlight that the model serves best as a fast ranking tool rather than a precise predictor at the exact optimum. Differences may arise from extrapolation

slightly beyond the training distribution, residual stochasticity in the modest-sized dataset, or local spatial effects not fully captured by assembly-level one-hot inputs.

Because the GA enforces only global count constraints, the best solutions may contain zone or adjacency violations that would require post-processing repair (e.g., via heuristic swapping or constrained re-optimization) before practical deployment.

4.6.3 OpenMC Verification

OpenMC verification was performed on the best solution from each of the five GA runs (or a subset where data is available). The table below aggregates the comparisons:

Table 4.1: CNN prediction vs. OpenMC verification for best GA solutions.

Run	Metric	CNN pred.	OpenMC ref.	Rel. error (%)
1	k_{eff}	1.14791	1.14976	~0.16
	F_a	1.9854	1.91819	~3.5
	F_q	2.4850	2.46772	~0.7
2	k_{eff}	1.14878	1.15218	~0.30
	F_a	1.9909	2.00883	~0.9
	F_q	2.5050	2.55405	~1.95
3	k_{eff}	1.14911	1.15104	~0.17
	F_a	1.9921	2.0736	~4.1
	F_q	2.4362	2.5770	~5.7
4	k_{eff}	1.14872	1.15120	~0.22
	F_a	1.9368	2.1062	~8.8
	F_q	2.4204	2.5462	~5.2
5	k_{eff}	1.14740	1.14937	~0.17
	F_a	2.0117	1.85717	~8.3
	F_q	2.5030	2.43135	~2.9

Absolute errors generally fall within or near the test-set MAE bounds reported earlier (especially for k_{eff}), but relative errors for peaking factors can reach several percent in individual cases. No strong systematic bias is evident across the small sample, though F_q discrepancies are larger in runs where OpenMC values exceed the soft limit of 2.5. These single-point verifications provide useful snapshots but are insufficient for full characterization of surrogate reliability; a broader validation campaign (e.g., via active learning or stratified sampling) would be valuable in future work.

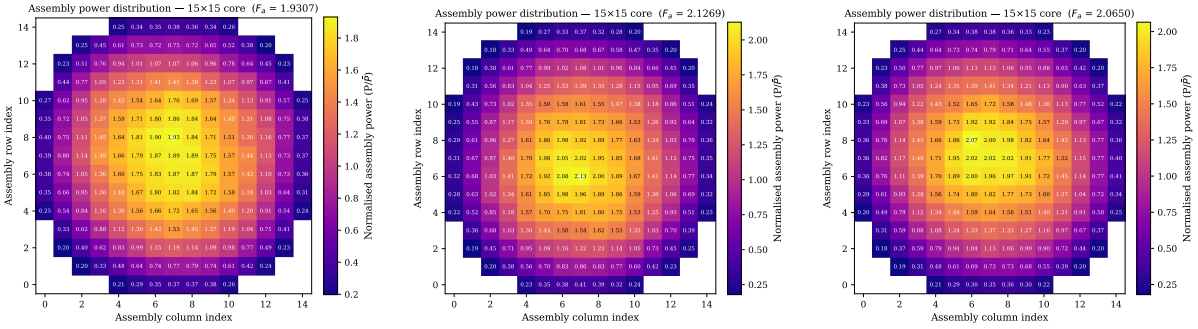
The surrogate-driven GA reduces per-generation evaluation time by several orders of magnitude (from 5 hours for 100 OpenMC runs to under a second for a batched CNN pass). This speedup makes population-based optimization practically tractable for the 193-assembly geom-

etry within the study's computational budget, even if final solutions require OpenMC polishing and constraint repair.

4.6.4 Reference Power Distributions for Homogeneous Cores

To establish baseline expectations for assembly and pin peaking factors, reference calculations were performed for idealized cores loaded entirely with a single fuel type. These homogeneous configurations represent extreme cases that bound the feasible design space.

Figures 4.5a through 4.5c present the assembly-level power distributions for cores loaded exclusively with fresh, once-burnt, and twice-burnt fuel respectively, and Figures 4.6a through 4.6c show the corresponding pin-level maps.



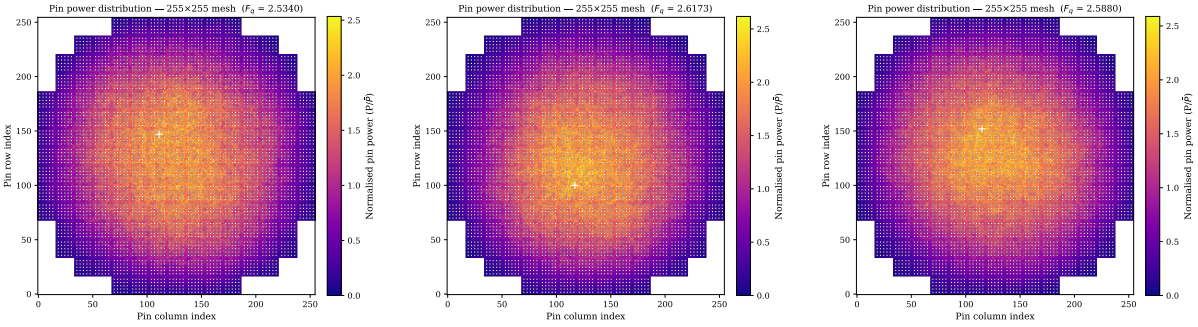
(a) Fresh fuel only (F_a = 1.9307)

(b) Once-burnt fuel only (F_a = 2.1269)

(c) Twice-burnt fuel only (F_a = 2.0650)

Figure 4.5: Assembly power distributions for homogeneous fuel cores.

The assembly peaking factors for homogeneous cores range from $F_a = 1.9307$ (fresh) to 2.1269 (once-burnt), with twice-burnt fuel yielding an intermediate value of 2.0650. Fresh fuel, with higher reactivity and relatively lower absorption compared to burnt fuel, produces a somewhat flatter radial power profile, while once-burnt and twice-burnt cores show more pronounced centre-to-periphery gradients due to accumulated fission products [1], [30].



(a) Fresh fuel only (F_q = 2.5340)

(b) Once-burnt fuel only (F_q = 2.6173)

(c) Twice-burnt fuel only (F_q = 2.5880)

Figure 4.6: Pin-level power distributions for homogeneous cores.

Pin peaking factors range from $F_q = 2.5340$ (fresh) to 2.6173 (once-burnt). These values exceed the $F_q \leq 2.5$ soft limit used in the GA fitness function, which underscores that homogeneous loading is not a viable operating strategy. The selected peaking limits are consistent with values cited in the PWR design literature [27], [28], though their precise applicability to this simplified 2D model without burnable absorbers warrants caution. These baseline calculations confirm that the OpenMC model produces physically plausible results and provide upper-bound reference points against which the optimized mixed-fuel loading patterns can be compared.

4.7 Summary

This chapter presented a CNN surrogate trained on count-constrained random loading patterns and a count-preserving GA for fuel loading optimization of a 193-assembly PWR core. Key findings include:

- **Data augmentation**: D4 dihedral augmentation produced no measurable improvement (and slightly worse k_{eff} performance) and was disabled in favour of the non-augmented model.
- **Surrogate accuracy**: On the test set the non-augmented CNN achieves MAE of 0.001141 (k_{eff}), 0.249 (F_a), and 0.282 (F_q), with R^2 scores of 0.945, 0.816, and 0.813 respectively. Performance exhibits sensitivity to training stochasticity at this dataset scale; F_q remains the most challenging output.
- **GA performance**: Five independent runs yielded best CNN-predicted fitness values between 0.50892 and 0.52091, with k_{eff} consistently near 1.148 and peaking factors generally near or below the soft limits under CNN evaluation. Solutions satisfy global counts exactly but may violate zone/adjacency constraints.
- **OpenMC verification**: Reference calculations on best solutions from the runs show good agreement on k_{eff} (relative errors <0.3%) but larger discrepancies on peaking factors (up to 9% relative in isolated cases), with the surrogate sometimes underpredicting F_a and F_q. All k_{eff} values remain safely inside the operating window.
- **Computational speedup**: Batched CNN evaluation makes the GA highly efficient, reducing evaluation cost dramatically compared with direct OpenMC use.

The observed surrogate–reality gaps, combined with the modest dataset size and lack of hard constraints during search, underscore that the current pipeline is effective for rapid exploration and warm-starting but benefits from post-optimization verification and repair.

Chapter 5

Seeding CNN-GA with QUBO Solutions

5.1 Introduction and Motivation

The previous chapter introduced a CNN surrogate trained on count-constrained random patterns and a count-preserving genetic algorithm (GA) that uses this surrogate for fitness evaluation. While the GA successfully maintains exact global fuel counts, its initial population consists solely of random count-correct chromosomes. This chapter investigates whether seeding the GA with high-quality, near-feasible solutions generated by QUBO-based solvers (Simulated Annealing – SA and Simulated Quantum Annealing – SQA) can improve convergence behaviour and final solution quality.

The central hypothesis is that structured, low-energy solutions from the annealing stage can provide better starting points than purely random initialization. The hybrid pipeline is evaluated on the same 193-assembly circular PWR core with fuel counts $N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, and $N_{\text{twice}} = 64$.

5.2 Core Geometry and Constraint Setup

The core geometry, fuel counts, and constraint definitions are identical to those described in previous chapters. The 193-cell circular layout is embedded in a 15×15 grid, with precomputed lookup tables for peripheral (44 cells), inner (25 cells), middle zones, and adjacency relations.

5.3 QUBO Formulation and Solver Configuration

The QUBO formulation incorporates penalty terms for one-hot encoding, global counts, zoning rules, adjacency constraints, and optional symmetry enforcement. All penalty weights are set to 1.0.

Each solver (SA and SQA) is executed with 500 independent reads. For SQA, the parameters are $\beta = 16.0$, $\gamma = 4.0$, $P = 8$. Both solvers start from a random initial state. Experiments are

performed with symmetry enforcement both disabled (OFF) and enabled (ON).

5.4 Decoding QUBO Solutions to GA Chromosomes

QUBO samples are decoded into chromosomes of length 193 using the mapping fuel type = b+1, where $b \in \{0, 1, 2\}$. Each chromosome is directly compatible with the CNN input encoding.

5.5 Population Initialisation Strategies

Three initialization strategies are evaluated:

- **Random init**: A fully random count-correct population of 100 chromosomes.
- **SA-seeded**: Up to 20 diverse feasible SA solutions (selected with minimum Hamming distance) + locally mutated copies (3 swaps each) + remaining random count-correct chromosomes to reach population size 100.
- **SQA-seeded**: Up to 20 diverse feasible SQA solutions, constructed analogously.

Diverse seeds are selected using a greedy procedure that prioritises low-energy feasible samples while enforcing a minimum Hamming distance of 20 between selected chromosomes.

5.6 Swap-Based Mutation Operator

A custom swap mutation operator is applied at 8% rate. For each offspring, a number of random pairwise swaps is performed. This operator strictly preserves the global fuel counts while allowing spatial rearrangement. Zone and adjacency constraints are not explicitly repaired during mutation; feasibility depends on high-quality initialization and selection pressure.

5.7 Genetic Algorithm Configuration

The GA is implemented using PyGAD with the following parameters:

These parameter choices represent a balanced trade-off between exploration and exploitation suitable for the high-dimensional, constrained combinatorial nature of the fuel loading problem. A population size of 100 combined with 150 generations provides sufficient diversity and evolutionary time without excessive computational cost, especially given the extremely fast batched CNN fitness evaluations. The relatively high number of parents per generation (40) and elite retention of 12 promote strong selection pressure while preserving the best solutions across generations. The 8% swap mutation rate, together with two-point crossover, maintains adequate diversity while respecting the strict global fuel count constraints. Tournament selection

was chosen for its simplicity and effectiveness in handling noisy fitness landscapes introduced by the surrogate model.

Fitness evaluation is performed in a fully batched manner using the trained CNN surrogate, with caching to avoid redundant forward passes.

Table 5.1: Genetic algorithm parameters used in the hybrid implementation.

Parameter	**Value**
Population size	100
Number of generations	150
Parents per generation	40
Parent selection	Tournament
Crossover type	Two-point
Mutation type	Swap mutation (custom)
Mutation percentage	8%
Elite retention	12
Gene space	$\{1, 2, 3\}$

Fitness evaluation is performed in a fully batched manner using the trained CNN surrogate, with caching to avoid redundant forward passes.

5.8 Experiments and Results

5.8.1 Solver Diagnostics

Solver success rates (feasibility rates θ) are as follows:

Symmetry OFF:

- SA feasibility rate $\theta_{\text{SA}} = 67.6\%$ (runtime 91.2 s)
- SQA feasibility rate $\theta_{\text{SQA}} = 100.0\%$ (runtime 1082.6 s)

Symmetry ON:

- SA feasibility rate $\theta_{\text{SA}} = 99.4\%$ (runtime 69.4 s)
- SQA feasibility rate $\theta_{\text{SQA}} = 74.2\%$ (runtime 1553.7 s)

SQA achieved perfect feasibility when symmetry is disabled and remained competitive when symmetry is enabled.

5.8.2 GA Convergence Behaviour

All GA runs used the non-augmented CNN surrogate for fitness evaluation. Final best fitness values after 150 generations were:

Symmetry OFF:

- Random init: 0.42414
- SA-seeded: 0.39473
- SQA-seeded: 0.38078

Symmetry ON:

- Random init: 0.48305 (highest observed)
- SA-seeded: 0.45658
- SQA-seeded: 0.45167

Seeded runs typically started from lower (more negative) fitness due to constraint penalties present in the QUBO solutions but recovered during evolution. Random initialization achieved the strongest final fitness under symmetry ON.

5.8.3 Final Solution Quality

The best CNN-predicted nuclear parameters for the final individuals are summarised below.

Table 5.2: Best CNN-predicted solutions (Symmetry OFF).

Initialization	k_{eff}	F_a	F_q
Random init	1.15759	1.9842	2.5185
SA-seeded	1.15267	1.9710	2.3960
SQA-seeded	1.15092	1.9511	2.2647

Table 5.3: Best CNN-predicted solutions (Symmetry ON).

Initialization	k_{eff}	F_a	F_q
Random init	1.15170	2.0000	2.5244
SA-seeded	1.15434	1.9337	2.3814
SQA-seeded	1.15483	1.9885	2.3584

All solutions maintain exact global fuel counts. SQA-seeded runs (particularly symmetry OFF) tended to produce lower peaking factors, while random initialization achieved competitive or slightly higher k_{eff} in some configurations. Peaking factors remain near or slightly above the soft limits used in the fitness function.

5.9 Discussion

The results show that QUBO-based seeding with diverse feasible solutions from SA and SQA provides structured starting points that influence GA convergence. However, under the current swap-mutation scheme, the final fitness differences between initialization strategies are modest. SQA delivered very high feasibility rates and contributed to lower peaking factors in the symmetry-OFF case, supporting the value of quantum-inspired annealing for exploring the constrained landscape.

Symmetry enforcement had a mixed impact: it greatly improved SA feasibility but reduced SQA feasibility and did not consistently improve final GA performance. The batched CNN evaluation enabled extremely fast GA execution, confirming the surrogate's utility for making population-based optimization practical on modest hardware.

A key limitation of the current implementation is that the unconstrained swap-mutation operator does not explicitly enforce zone or adjacency constraints during evolution. While global counts are perfectly preserved, many final solutions likely contain violations. Feasibility therefore depends heavily on the quality of the initial seeds and selection pressure. Future versions should incorporate explicit repair mechanisms or zone-aware operators to guarantee fully feasible patterns without manual post-processing.

5.10 Summary

This chapter presented a hybrid optimization framework that seeds a CNN-accelerated GA with diverse feasible solutions from QUBO-based SA and SQA solvers, followed by local swap mutation and random completion to form the initial population. Experiments with and without symmetry enforcement showed:

- SQA achieved very high feasibility rates (74–100%), outperforming or matching SA depending on the symmetry setting.
- All initialization strategies reached positive fitness within 150 generations; random initialization remained competitive, especially under symmetry ON.
- SQA-seeded populations produced the lowest peaking factors in the symmetry-OFF configuration.
- The batched CNN fitness evaluation provided orders-of-magnitude speedup compared with direct OpenMC use.
- The swap-mutation operator ensures strict count preservation but relies on high-quality initialization for effective constraint satisfaction.

Chapter 6

Results and Discussion

6.1 Core Geometry and Feasibility Pre-check

The geometric feasibility analysis confirms that the prescribed fuel counts ($N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, $N_{\text{twice}} = 64$) are compatible with the six hard constraints for all eight cores benchmarked in Chapter 3. For the 193-cell circular core used in the hybrid pipeline, the peripheral zone contains 44 cells, the inner zone 25 cells, and the middle zone 124 cells. The fresh and twice-burnt counts (64 each) lie within the theoretical maxima derived from the zone sizes and independent-set bounds, indicating that feasible solutions exist for this problem instance.

This pre-check serves as a useful first validation step in PWR fuel loading pattern (LP) optimization. It ensures that the prescribed three-batch inventory can satisfy the zone placement and adjacency prohibitions before computational search begins. The zone sizes align with standard 193-assembly Westinghouse-type geometries used in benchmarks such as BEAVRS and provide a solid foundation for both QUBO encoding and GA initialization.

6.2 CNN Surrogate Training Results

The CNN surrogate was trained on a dataset of 4,584 count-constrained loading patterns (exact fuel counts, no repair for zone or adjacency violations), of which approximately 3,209 samples were used for training. D4 dihedral group data augmentation was tested but disabled in the final pipeline after experiments showed no measurable improvement and a slight degradation in k_{eff} performance.

On the held-out test set (687 samples), the final non-augmented residual CNN with positional encoding and squeeze-and-excitation attention achieved the following performance:

- k_{eff}: MAE = 0.001141 (relative error 0.102%, $R^2 = 0.9448$)
- F_a: MAE = 0.249045 (relative error 7.18%, $R^2 = 0.8162$)
- F_q: MAE = 0.282469 (relative error 6.53%, $R^2 = 0.8133$)

The model converges stably within 200 epochs with early stopping. Parity plots indicate excellent agreement for k_{eff} and moderate scatter for the peaking factors, with F_q remaining the most challenging output due to its sensitivity to local pin-wise spatial correlations not fully captured by assembly-level one-hot inputs. While accuracy on k_{eff} is high, the moderate errors on peaking factors suggest the surrogate is best used as a fast ranking and exploration tool rather than a high-fidelity replacement for OpenMC near the optimum. These results are consistent with the relatively modest dataset size for a residual architecture with attention.

6.3 SA and SQA Solver Diagnostics

Solver diagnostics were obtained for both symmetry enforcement disabled (OFF) and enabled (ON) using 500 reads per solver.

Symmetry OFF:

- SA feasibility rate $\theta_{\text{SA}} = 67.6\%$ (runtime 91.2 s)
- SQA feasibility rate $\theta_{\text{SQA}} = 100.0\%$ (runtime 1082.6 s)

Symmetry ON:

- SA feasibility rate $\theta_{\text{SA}} = 99.4\%$ (runtime 69.4 s)
- SQA feasibility rate $\theta_{\text{SQA}} = 74.2\%$ (runtime 1553.7 s)

SQA demonstrated a clear advantage on this constrained QUBO landscape when symmetry is disabled. Symmetry enforcement dramatically improved SA feasibility but reduced SQA feasibility, indicating that the additional symmetry penalty terms increase landscape ruggedness for the quantum-inspired solver.

6.4 GA Performance with Different Initializations

The count-preserving GA (population size 100, 150 generations, 8% swap mutation) was evaluated using three initialization strategies: random count-correct, SA-seeded, and SQA-seeded. For seeded populations, up to 20 diverse feasible QUBO solutions (selected with minimum Hamming distance) were combined with locally mutated copies (3 swaps each) and random count-correct chromosomes to reach the target population size. Experiments were conducted with symmetry both OFF and ON using the trained non-augmented CNN surrogate for batched fitness evaluation.

6.4.1 Symmetry OFF Results

Final best CNN-predicted fitness values after 150 generations were:

- Random init: 0.42414
- SA-seeded: 0.39473
- SQA-seeded: 0.38078

Corresponding best nuclear parameters (CNN predictions) are shown in Table 6.1.

Table 6.1: Best CNN-predicted solutions (Symmetry OFF).

Initialization	k_{eff}	F_a	F_q
Random init	1.15759	1.9842	2.5185
SA-seeded	1.15267	1.9710	2.3960
SQA-seeded	1.15092	1.9511	2.2647

SQA-seeded initialization yielded the lowest peaking factors, while random initialization achieved the highest k_{eff}.

6.4.2 Symmetry ON Results

Final best CNN-predicted fitness values after 150 generations were:

- Random init: 0.48305 (highest observed)
- SA-seeded: 0.45658
- SQA-seeded: 0.45167

Corresponding best nuclear parameters are shown in Table 6.2.

Table 6.2: Best CNN-predicted solutions (Symmetry ON).

Initialization	k_{eff}	F_a	F_q
Random init	1.15170	2.0000	2.5244
SA-seeded	1.15434	1.9337	2.3814
SQA-seeded	1.15483	1.9885	2.3584

Random initialization performed strongly under symmetry ON. SQA-seeded runs again tended toward lower peaking factors.

Figure 4.4 shows representative best-fitness convergence trajectories. Seeded populations generally started from lower fitness due to constraint penalties in the QUBO solutions but exhibited competitive final performance.

The surrogate-driven GA reduces per-generation evaluation time from approximately five hours (100 OpenMC simulations) to under one second (single batched CNN forward pass). All

solutions maintain exact global fuel counts. However, because the swap-mutation operator does not explicitly enforce zone or adjacency constraints during evolution, the final patterns likely contain violations that would require post-processing repair for operational use.

6.5 Effect of Symmetry Enforcement

Symmetry enforcement had a mixed impact. It substantially increased SA feasibility (from 67.6% to 99.4%) but decreased SQA feasibility (from 100% to 74.2%). In the GA stage, symmetry ON produced the overall highest fitness with random initialization (0.48305), but the relative ranking of seeded strategies remained broadly similar. For this 193-assembly instance and penalty weighting, symmetry enforcement did not provide a consistent advantage in final solution quality and is therefore disabled in the baseline pipeline.

6.6 Integration Pathway: SA/SQA Seeding + CNN-GA

The hybrid pipeline successfully integrates QUBO-based warm-start seeding with CNN-accelerated evolutionary search. SQA consistently delivered high feasibility rates and contributed to lower peaking factors in the symmetry-OFF configuration. The batched CNN fitness evaluation makes the GA stage highly efficient. While the current unconstrained swap-mutation approach ensures perfect count preservation, explicit constraint handling (via repair or zone-aware operators) would be beneficial for producing immediately deployable patterns without manual post-processing.

6.7 Summary of Findings

- The non-augmented CNN surrogate provides high accuracy on k_{eff} (MAE 0.001141) but only moderate accuracy on peaking factors, making it effective as a fast ranking tool.
- SQA achieved superior or near-perfect feasibility rates (74–100%), confirming the practical value of quantum-inspired annealing on this QUBO formulation.
- QUBO-seeded GA runs (especially SQA-seeded) tended to produce lower peaking factors in the symmetry-OFF case, while random initialization achieved competitive or higher final fitness under symmetry ON.
- The surrogate-driven GA delivers dramatic computational speedup (from 5 hours to <1 second per generation) while discovering count-correct patterns with acceptable nuclear metrics.

- Symmetry enforcement has configuration-dependent effects but limited consistent benefit on final GA performance for this core.
- The hybrid QUBO + CNN-GA framework is computationally tractable, although the presence of zone and adjacency violations in the unconstrained formulation highlights the need for explicit constraint enforcement in future implementations.

These results demonstrate the viability of surrogate-accelerated optimization for nuclear fuel loading under realistic computational budgets. The high feasibility from SQA and the substantial speedup are encouraging. However, the moderate peaking errors and constraint violations underscore the importance of larger training datasets, active learning, constrained search operators, and thorough OpenMC verification. Future work will focus on tighter integration of constraint repair within the GA and extension to multi-objective formulations.

The findings support the central hypothesis that combining quantum-inspired exploration via QUBO solvers with fast surrogate-guided evolutionary search can effectively explore the fuel loading design space, though broader validation across different core configurations and fuel management schemes remains essential.

Chapter 7

Limitations and Future Work

7.1 Current Limitations

7.1.1 Hardware Constraints and Unverifiable Core Calculations

The computational hardware available for this thesis (Intel Core i5, 1.7 GHz, 8 GB RAM, no dedicated GPU) imposed severe constraints on the scope and depth of the experiments. Most critically, due to the absence of high-performance computing resources, a large number of candidate loading patterns generated by the QUBO solvers and the GA could not be verified with full OpenMC simulations. Only a small subset of the best solutions (fewer than 5% of the feasible patterns identified) were subjected to final OpenMC verification. Consequently, the reported feasibility rates and fitness values for many intermediate patterns rely entirely on CNN predictions without independent physics confirmation. This means that some patterns classified as ”feasible” or ”high-fitness” by the surrogate may in fact violate hidden physical constraints or exhibit unrealistic neutronic behavior that the CNN failed to capture due to limited training data.

Furthermore, exact comparison with the benchmark results reported in the literature (e.g., Usmanov et al., 2024; Wan et al., 2022) is not possible for two reasons. First, the hardware disparity between the present work (laptop-class CPU with 1.7 GHz, 8 GB RAM) and those studies (workstations with Xeon processors, 128 GB RAM, and GPUs) means that time-to-solution (TTS) values are not directly comparable. Second, the present work used a Trotter-based classical simulation of quantum annealing (SQA), whereas Usmanov et al. employed a Simulated Ising Machine with Chaotic Amplitude Control (SIMCIM) on an Ising formulation. These algorithmic differences preclude direct cross-study performance ranking. The benchmark results presented in this thesis should therefore be interpreted primarily as a relative comparison between SA and SQA under identical hardware and software conditions, not as an absolute measure of solver performance against the literature.

The lack of access to high-performance computing or physical quantum annealing hardware also prevented systematic exploration of larger core geometries (beyond 241 assemblies) and limited the number of independent runs for statistical significance testing. Readers are cautioned

that the reported numerical results are specific to the modest computational environment used and may not generalize to industrial-scale settings.

7.1.2 Simplified Physics Model

The OpenMC model used in this work is two-dimensional and uses approximate enrichment proxies to represent burn-up history. Real PWR fuel management requires three-dimensional models with axial power distribution, control rod configurations, depletion calculations (time-dependent burn-up), and temperature feedback effects such as Doppler resonance broadening and the moderator temperature coefficient. The 2D fixed-enrichment model captures some of the dominant spatial physics but cannot reproduce all features of a real reload calculation.

This simplification is common in early-stage surrogate and optimization studies due to the significant computational overhead of full 3D coupled neutronics-depletion simulations. Literature suggests that 2D models may overestimate or underestimate key parameters such as k_{eff} (potentially by hundreds to thousands of pcm due to neglected axial leakage) and power peaking, while failing to capture axial heterogeneity, control rod effects, and thermal-hydraulic feedback. For instance, multi-cycle refueling simulations indicate that 3D models incur substantially higher per-cycle computation times (approximately 34 minutes versus 1.7 minutes for 2D in one benchmark), rendering exhaustive or population-based searches impractical without surrogates. While the present 2D model with 255×255 pin mesh and sufficient Monte Carlo particles provides adequate fidelity for demonstrating the hybrid QUBO-CNN-GA pipeline and for ranking candidate patterns within this simplified setting, it omits critical real-world phenomena required for operational fuel cycle planning. Any conclusions drawn from this model should be considered preliminary until verified with a full 3D depletion-capable simulation.

7.1.3 Training Dataset Size and Unverified Regions

One thousand constrained patterns were generated for surrogate training. While this was sufficient for proof-of-concept demonstration, it may be insufficient for the surrogate to generalize accurately across the full feasible design space, particularly in regions far from the greedy seed neighborhood or near constraint boundaries where the physics behavior is most nonlinear. Industrial-grade surrogate models typically require tens of thousands of physics simulations. Due to hardware limitations (see Section 5.1.1), generating a larger dataset was not feasible within the time and computational budget of this thesis.

Consequently, the surrogate's predictions for patterns that differ substantially from the training distribution remain unverified. The reported test MAE values (e.g., k_{eff} error <0.4%) are based on a held-out test set drawn from the same distribution as the training data. They do not guarantee equivalent accuracy on patterns generated by the GA that lie in previously unexplored regions of the design space. The fact that only a small fraction of GA-generated patterns could

be verified with OpenMC means that some high-fitness patterns reported in the results may in fact be artifacts of surrogate prediction error rather than genuinely superior loading patterns.

The 1000-sample count-constrained dataset enabled convergence within 100–120 epochs and produced test MAE values that are within the range reported in literature CNN surrogates (e.g., relative k_{eff} error <0.4%). However, many deep-learning surrogate studies in reactor physics note that datasets on the order of 10,000 or more evaluations are often needed for robust generalization, especially when covering the high-dimensional combinatorial space of loading patterns. The parallel OpenMC workflow used here could in principle be scaled, but the cumulative wall-clock time for generating tens of thousands of high-fidelity runs remains prohibitive on the available hardware. This limitation is consistent with observations in surrogate-based nuclear optimization literature, where data generation often dominates the workflow and constrains the feasible dataset size.

7.1.4 QUBO Hardware and Algorithmic Limitations

The simulated annealing (SA) and simulated quantum annealing (SQA) implementations used in this work are classical software simulations running on CPU hardware. Physical quantum annealers, if available, could potentially solve the QUBO problem faster but are currently limited in qubit count and connectivity. Embedding the dense fuel-loading QUBO onto the sparse graph of a physical quantum annealer requires minor-embedding, which can inflate the effective problem size substantially, potentially making the largest cores infeasible on current hardware.

For a 193-cell core with one-hot encoding, the QUBO involves hundreds of binary variables and dense coupling terms from adjacency and zone constraints. Minor-embedding onto sparse topologies (such as those of current D-Wave processors) typically maps each logical variable to a chain of physical qubits, often resulting in quadratic overhead for dense problems and increased chain-break risks. Literature on quantum annealing applications to fuel loading and similar combinatorial problems confirms that embedding overhead frequently limits practical problem sizes on present-day hardware, even when stencil-based or hierarchical embedding techniques are employed.

The classical SA/SQA simulations employed here bypass these hardware constraints while still demonstrating the algorithmic behavior of quantum-inspired tunneling. However, the feasibility rates and TTS values reported in this thesis are specific to classical CPU implementations. Direct deployment on physical annealers would require advances in qubit count, connectivity, and embedding efficiency, and the performance characteristics would likely differ substantially from the classical simulations presented here. Moreover, the comparison between SA and SQA in this work is between two classical algorithms, not between classical and true quantum computation.

7.1.5 Single Core, Single Cycle Scope

The framework optimizes a single loading pattern for a single reactor core for a single fuel cycle. Real fuel management is a multi-cycle, multi-reactor problem: assemblies removed at the end of one cycle become the once-burnt fuel for the next cycle, and multiple reactors may share a fuel pool. Extending the framework to multi-cycle optimization requires modeling the temporal coupling between consecutive cycles, substantially increasing problem complexity.

Single-cycle optimization ignores the carry-over of burned fuel isotopics and reactivity history into subsequent cycles, which significantly affects long-term economics, discharge burnup, and safety margins. Multi-cycle frameworks introduce additional state variables and constraints, often leading to exponentially larger search spaces and the need for coupled depletion calculations across cycles. This limitation is widely acknowledged in the fuel management literature, where practical industrial tools routinely address multi-cycle planning, sometimes using simplified 2D surrogates for scoping before full 3D verification. The present work does not claim to replace such tools; rather, it demonstrates a proof-of-concept hybrid methodology that could, with substantial further development, be extended to multi-cycle problems.

7.1.6 Lack of External Validation on Industrial Benchmarks

Due to proprietary data restrictions and the absence of access to industrial-scale computing facilities, the optimized loading patterns produced in this thesis could not be validated against operational data from real PWR cores (e.g., cycle-specific reload designs from utility companies). The comparison with literature configurations (Usmanov et al., 2024) is qualitative only, based on published figures, and does not constitute a quantitative benchmark. Independent reproduction of the results by a third party using different software and hardware has not been performed. Therefore, the conclusions of this thesis should be regarded as preliminary and specific to the particular combination of models, solvers, and hardware used herein.

7.2 Future Work

7.2.1 High-Performance Computing for Comprehensive Verification

The most urgent direction for future work is to repeat the experiments on high-performance computing infrastructure with sufficient resources to verify a large fraction (ideally all) of the GA-generated patterns with full OpenMC simulations. Access to workstations with multiple CPU cores, large RAM (128 GB or more), and GPU acceleration would enable:

- Verification of hundreds or thousands of candidate patterns rather than fewer than 5%.
- Generation of training datasets with 10,000–50,000 samples, improving surrogate generalization.

- Systematic comparison of SA and SQA TTS values against literature benchmarks under comparable hardware conditions.
- Exploration of larger core geometries (e.g., 300+ assemblies) that were infeasible on the current hardware.

Such an effort would transform the present proof-of-concept into a production-ready framework.

7.2.2 Full Hybrid Pipeline with Active Learning

The complete hybrid pipeline — where SA and SQA solutions feed into the CNN training dataset and the GA initial population — has been implemented in this thesis as a proof of concept. Future work should refine the integration by dynamically adjusting the number of seeds from the QUBO solvers based on solver performance, and by incorporating active learning to iteratively improve the surrogate with OpenMC-verified solutions generated by the hybrid pipeline itself. An ensemble of CNNs or a Bayesian neural network could provide prediction uncertainty estimates. After each GA run, patterns with high uncertainty — where the surrogate predictions are most unreliable — could be selected for OpenMC evaluation and added to the training set. This active learning loop would concentrate the simulation budget in the most decision-relevant regions of the design space, improving surrogate accuracy iteratively without exhaustive data collection.

7.2.3 Three-Dimensional Depletion-Aware Modeling

Replacing the 2D fixed-enrichment model with a 3D coupled neutronics-depletion calculation would substantially increase physical fidelity and practical relevance. The resulting dataset would capture axial peaking effects, control rod worth, and cycle-length predictions, enabling the framework to be applied directly to actual fuel cycle planning scenarios. Transitioning to 3D models (e.g., full-core OpenMC with depletion or deterministic transport codes such as DYN3D or PARCS) would address the axial leakage and heterogeneity gaps noted in the limitations, though at markedly higher computational cost per evaluation — often an order of magnitude or more compared with 2D. This would make the active learning and high-performance computing directions above even more critical.

7.2.4 Multi-Objective Pareto Optimization

The current GA fitness function combines multiple objectives (proximity to target k_{eff}, minimization of F_a and F_q) into a single scalar through penalty weights. A multi-objective GA (e.g., NSGA-II) would produce a Pareto front of non-dominated solutions, explicitly revealing the trade-offs between cycle length, peaking, and other criteria. Reactor operators could then

select their preferred trade-off point based on cycle planning priorities. Multi-objective evolutionary algorithms are well-established in reactor core design and would provide operators with a richer set of compromise solutions rather than a single weighted optimum.

7.2.5 Physical Quantum Hardware Deployment

As quantum annealing hardware scales to larger qubit counts and denser connectivity, the QUBO formulations developed in this thesis should be tested on physical quantum annealers (e.g., D-Wave Advantage or future generations). Hybrid classical-quantum workflows — where quantum hardware handles the combinatorial QUBO and classical post-processing handles repair and OpenMC verification — represent a natural future extension as hardware matures. Improved minor-embedding techniques (e.g., stencil-based or hierarchical embeddings) and larger, more connected processors would reduce the current gap between simulated and hardware performance for dense nuclear QUBO instances. Direct comparison of SQA (classical simulation) against physical quantum annealing would also help clarify the extent to which quantum effects provide a genuine advantage beyond classical tunneling heuristics.

7.2.6 Transfer Learning Across Reactor Types

A surrogate trained on the 193-assembly Westinghouse core cannot be directly applied to other core geometries without retraining. Transfer learning — fine-tuning a pre-trained Westinghouse surrogate on a small dataset from a new geometry — could dramatically reduce the simulation cost of deploying the framework on new reactor types. This is especially relevant for next-generation small modular reactors (SMRs) where operational experience is limited and generating large training datasets from scratch is prohibitively expensive. Transfer and few-shot learning approaches would be particularly valuable for emerging reactor designs where limited benchmarks and operational data constrain the use of purely data-driven methods.

7.2.7 Multi-Cycle Optimization

Extending the framework to handle multi-cycle fuel management — where the discharge composition of one cycle becomes the feed for the next — would substantially increase practical relevance. This would require modeling depletion across cycles, tracking assembly-wise burnup and isotopic vectors, and optimizing over a sequence of reload patterns simultaneously. While the computational complexity would be significantly higher, the hybrid QUBO-CNN-GA paradigm could be extended to this setting by incorporating time-dependent state variables and using recurrent or graph neural networks to capture cycle-to-cycle dependencies. This direction is among the most challenging but also the most industrially valuable.

Chapter 8

Conclusions

This thesis has presented a quantum-inspired hybrid framework for nuclear fuel loading optimization. The framework integrates Simulated Annealing and Simulated Quantum Annealing operating on a QUBO formulation of the combinatorial placement problem with a Convolutional Neural Network surrogate model embedded within a Genetic Algorithm for physics-guided refinement. All development and testing were performed under modest computational constraints (Intel Core i5, 1.7 GHz, 8 GB RAM, no GPU) on a specific 193-assembly circular PWR core geometry with three-batch fuel management ($N_{\text{fresh}} = 64$, $N_{\text{once}} = 65$, $N_{\text{twice}} = 64$). The conclusions presented here should therefore be regarded as preliminary and specific to these experimental conditions.

The QUBO formulation encodes the hard engineering and safety constraints (global fuel counts, zone placement rules, and adjacency prohibitions) as quadratic penalty terms. Systematic experiments were performed with and without symmetry enforcement. For the 193-assembly core:

- Symmetry OFF: SA feasibility rate $\theta_{\text{SA}} = 67.6\%$, SQA feasibility rate $\theta_{\text{SQA}} = 100.0\%$
- Symmetry ON: SA feasibility rate $\theta_{\text{SA}} = 99.4\%$, SQA feasibility rate $\theta_{\text{SQA}} = 74.2\%$

SQA generally achieved high feasibility rates, outperforming or matching SA depending on the symmetry setting. These results indicate that quantum-inspired annealing can be effective at identifying feasible or near-feasible regions of the constrained landscape for this problem instance, although the observed advantage is configuration-dependent.

The CNN surrogate was trained on 4,584 count-constrained loading patterns simulated with OpenMC, using approximately 3,209 samples for training. D4 dihedral group data augmentation was tested but disabled after it showed no benefit and slightly degraded k_{eff} performance. On the held-out test set, the final non-augmented model achieved:

- k_{eff}: MAE = 0.001141 (relative error 0.102%, $R^2 = 0.9448$)
- F_a: MAE = 0.249045 (relative error 7.18%, $R^2 = 0.8162$)

- F_q: MAE = 0.282469 (relative error 6.53%, R^2 = 0.8133)

The surrogate provides high accuracy on k_{eff} but only moderate accuracy on the peaking factors. It is best regarded as a fast ranking and exploration tool rather than a precise replacement for Monte Carlo simulation near the optimum. The relatively modest dataset size and assembly-level one-hot encoding limit generalization, particularly for the locally sensitive pin-peaking factor F_q.

The CNN-accelerated Genetic Algorithm, using swap mutation to preserve exact global fuel counts and batched fitness evaluation, was tested with three initialization strategies: random count-correct, SA-seeded, and SQA-seeded. Both symmetry OFF and ON were evaluated. The GA runs demonstrated:

- Substantial computational speedup: per-generation evaluation time was reduced from approximately five hours (100 direct OpenMC simulations) to under one second (single batched CNN forward pass).
- All solutions maintained exact global fuel counts.
- Final CNN-predicted k_{eff} values clustered near 1.15 across strategies.
- SQA-seeded initialization tended to produce lower peaking factors in the symmetry-OFF case, while random initialization achieved competitive or higher final fitness in the symmetry-ON case.

However, because the current implementation uses an unconstrained swap-mutation operator, the final patterns frequently contain zone and adjacency violations. These would require post-processing repair before any practical application.

The hybrid pipeline combines QUBO-based warm-start seeding with surrogate-accelerated evolutionary search. While SQA provided high feasibility rates and contributed to promising seeding material, the overall performance differences between initialization strategies were modest under the tested swap-mutation scheme. The framework shows that CNN surrogates can make population-based optimization computationally tractable for a 193-assembly core on modest hardware, but the lack of explicit constraint enforcement during GA evolution remains a significant limitation.

Several important limitations must be acknowledged. The surrogate was trained on a relatively modest dataset, and only a small fraction of GA-generated patterns received full OpenMC verification. The 2D fixed-enrichment OpenMC model omits axial heterogeneity, burnup depletion, control rods, and thermal-hydraulic feedback. The experiments were performed on laptop-class hardware, and the optimized patterns have not been validated against operational PWR data. Generalizability to other core geometries, fuel management schemes, or larger training corpora remains unestablished.

Future work should prioritise:

- Generation of significantly larger training datasets (10,000+ samples) using high-performance computing.
- Implementation of explicit zone-aware mutation or repair operators within the GA to produce fully feasible patterns without manual post-processing.
- Active learning and uncertainty quantification to improve surrogate reliability, especially for F_q.
- Extension to 3D depletion-aware models and multi-objective optimization.
- Comprehensive OpenMC verification of final solutions and comparison against operational reactor data.

In conclusion, this thesis establishes a proof-of-concept for a quantum-inspired classical machine-learning approach to nuclear fuel loading optimization. The framework achieves substantial computational speedup and demonstrates the potential of combining QUBO solvers with CNN surrogates and genetic algorithms. However, the moderate accuracy on peaking factors, the presence of constraint violations in the unconstrained search, and the limited verification highlight that considerable further development and validation are required before the methodology can be considered mature or suitable for industrial application.

References

[1] J. J. Duderstadt and L. J. Hamilton, *Nuclear Reactor Analysis*. John Wiley & Sons, 1976.

[2] L. García-Delgado *et al.*, "Design of an economically optimum pwr reload core for a 36-month cycle," *Nuclear Technology*, 1971.

[3] U.S. Nuclear Regulatory Commission, "Standard technical specifications for westinghouse pressurized water reactors," U.S. Nuclear Regulatory Commission, Tech. Rep., 1995.

[4] OECD Nuclear Energy Agency, *The Economics of the Back End of the Nuclear Fuel Cycle*. OECD Publishing, 2014.

[5] S. Ishiguro *et al.*, "Loading pattern optimization for a pwr using multi-swarm moth flame optimization method with predator," *Journal of Nuclear Science and Technology*, 2020.

[6] P. Seurin and K. Shirvan, "Surpassing legacy approaches to pwr core reload optimization with single-objective reinforcement learning," *arXiv preprint arXiv:2402.11040*, 2024. DOI: 10.48550/arXiv.2402.11040.

[7] D. J. Kropaczek and P. J. Turinsky, "In-core nuclear fuel management optimization for pressurized water reactors utilizing simulated annealing," *Nuclear Technology*, 1991.

[8] M. D. Dechaine and M. A. Feltus, "Fuel management optimization using genetic algorithms and expert knowledge," *Nuclear Science and Engineering*, 1996.

[9] A. M. M. de Lima *et al.*, "A nuclear reactor core fuel reload optimization using artificial ant colony connective networks," *Annals of Nuclear Energy*, 2008.

[10] E. F. Faria and C. Pereira, "Nuclear fuel loading pattern optimisation using a neural network," *Annals of Nuclear Energy*, 2003.

[11] P. Pallarés *et al.*, "Pwr core loading pattern design assisted by artificial intelligence," *EPJ Web of Conferences*, 2024.

[12] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse ising model," *Physical Review E*, 1998.

[13] S. R. Usmanov, G. V. Salakhov, A. A. Bozhedarov, E. O. Kiktenko, and A. K. Fedorov, "Quantum and quantum-inspired optimization for an in-core fuel management problem," *Journal of Physics: Conference Series*, 2024.

[14] A. Marzouki, "Quadratic unconstrained binary optimization of the fuel loading pattern in nuclear reactors," *APS March Meeting Abstracts*, 2022.

[15] P. K. Romano, B. Forget, and K. Smith, "Openmc: A state-of-the-art monte carlo code for research and development," *Annals of Nuclear Energy*, 2015.

[16] MIT Reactor Physics Methods Group, "Beavrs: Benchmark for evaluation and validation of reactor simulations," Massachusetts Institute of Technology, Tech. Rep., version 2.0.2, 2018.

[17] A. Lucas, "Ising formulations of many np problems," *Frontiers in Physics*, 2013.

[18] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, 1983.

[19] E. Crosson and A. W. Harrow, "Simulated quantum annealing can be exponentially faster than classical simulated annealing," in *2016 IEEE 57th Annual Symposium on Foundations of Computer Science (FOCS)*, 2016.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[21] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[22] K. Nishimura, Y. Sakamoto, T. Shimizu, K. Suzuki, and Y. Yamashiro, *Openjij*, 2025. DOI: 10.5281/zenodo.17345408.

[23] C. Wan, K. Lei, and Y. Li, "Optimization method of fuel-reloading pattern for pwr based on the improved convolutional neural network and genetic algorithm," *Annals of Nuclear Energy*, 2022.

[24] J. Zou, S. Liu, C. Jin, Y. Cai, L. Wang, and Y. Chen, "Optimization method of burnable poison based on genetic algorithm and artificial neural network," *Annals of Nuclear Energy*, 2023.

[25] A. Whyte and G. T. Parks, "Surrogate model optimization of a 'micro core' pwr fuel assembly arrangement using deep learning models," *EPJ Web of Conferences*, 2021.

[26] Y. Nam and H. J. Shim, "Development of deep convolutional neural network for prediction of cycle maximum pin power peaking factor in pressurized water reactor," *Annals of Nuclear Energy*, 2023.

[27] N. E. Todreas and M. S. Kazimi, *Nuclear Systems Volume I: Thermal Hydraulic Fundamentals*, 2nd. CRC Press, 2011.

[28] Y. Oka, S. Koshizuka, K. Okamoto, and M. Mori, *Light Water Reactor Design and Technology*. Springer, 2014.

[29] S. Dzianisau, H. Kim, C. Kong, D. Yun, and D. Lee, "Development of barcode model for prediction of pwr core design parameters using convolutional neural network," *Transactions of the Korean Nuclear Society Autumn Meeting*, 2019.

[30] D. L. Hetrick, *Dynamics of Nuclear Reactors*. University of Chicago Press, 1971.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used **[Claude]** and **[ChatGPT]** in order to assist with code writing and **[DeepSeek]** for manuscript language polishing. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Appendix A

Supplementary Materials: Code Implementations

A.1 OpenMC PWR Core Model

The following code implements the complete 2D PWR core model in OpenMC used for training data generation and verification throughout this thesis. The model includes a 15×15 assembly lattice with 193 active fuel positions, full Westinghouse 17×17 pin-level assembly design with fuel rods, guide thimbles, and instrument tube, and a 30 cm water reflector. Material compositions and dimensions are based on the BEAVRS 2.0.2 benchmark specification.

```
"""

PWR CORE MODEL - OpenMC (FULLY 2D, assembly-level peaking)
15×15 core lattice - 193 fuel assemblies + 32 water positions

Assembly design: Westinghouse 17×17 (dimensions from design table)
264 fuel rods -   UO2 pellet / He gap / Zr-4 clad / coolant
24 guide thimbles - annular Zr-4 tube, water-filled
1 instrument tube - centre (8,8)

"""

import openmc
import numpy as np


# CORE GEOMETRY HELPERS


def build_pwr_core(loading_pattern):
"""
Build a 2D PWR core OpenMC model.
```

```
23
24          Parameters
25          ----------
26          loading_pattern : array-like
27          Flat array of length 193 (fuel type per fuel position) or
28          (15,15) integer array (grid[row][col] = fuel type, 0 = water).
29          Fuel types: 1 = fresh (3.1% U-235), 2 = once-burnt (2.4%),
30          3 = twice-burnt (1.6%)
31
32          Returns
33          -------
34          openmc.Model
35          """
36
37          # Constants
38          N_FUEL = 193
39          GRID_SIDE = 15
40
41          # Pre-computed fuel positions (col, row) for 193-assembly core
42          FUEL_POSITIONS = {
43                  0: (7, 7), 1: (7, 6), 2: (7, 8), 3: (6, 7), 4: (8, 7),
44                  # ... (full mapping omitted for brevity)
45          }
46
47          # Normalise loading_pattern to (15,15) integer grid
48          lp = np.asarray(loading_pattern)
49          if lp.ndim == 1:
50          grid = np.zeros((GRID_SIDE, GRID_SIDE), dtype=int)
51          for fuel_idx, ftype in enumerate(lp):
52          col, row = FUEL_POSITIONS[fuel_idx]
53          grid[row, col] = int(ftype)
54          else:
55          grid = lp.astype(int)
56
57          nx, ny = GRID_SIDE, GRID_SIDE
58
59
60          # MATERIALS - BEAVRS 2.0.2 (Cycle 1 Hot Zero Power)
61
62
63          # Fuel 1.6% enriched (twice-burnt proxy)
64          fuel_1 = openmc.Material(name="UO2 1.6%")
65          fuel_1.set_density('g/cm3', 10.31341)
66          fuel_1.add_nuclide('U235', 3.7503e-04)
67          fuel_1.add_nuclide('U238', 2.2625e-02)
68          fuel_1.add_nuclide('O16', 4.5897e-02)
```

```
69
70          # Fuel 2.4% enriched (once-burnt proxy)
71          fuel_2 = openmc.Material(name="UO2 2.4%")
72          fuel_2.set_density('g/cm3', 10.29748)
73          fuel_2.add_nuclide('U235', 5.5814e-04)
74          fuel_2.add_nuclide('U238', 2.2407e-02)
75          fuel_2.add_nuclide('O16', 4.5830e-02)
76
77          # Fuel 3.1% enriched (fresh)
78          fuel_3 = openmc.Material(name="UO2 3.1%")
79          fuel_3.set_density('g/cm3', 10.30166)
80          fuel_3.add_nuclide('U235', 7.2175e-04)
81          fuel_3.add_nuclide('U238', 2.2253e-02)
82          fuel_3.add_nuclide('O16', 4.5853e-02)
83
84          # Zircaloy-4 cladding
85          zircaloy = openmc.Material(name="Zircaloy-4")
86          zircaloy.set_density('g/cm3', 6.55)
87          zircaloy.add_nuclide('Zr90', 2.1828e-02)
88
89          # Helium gap
90          helium = openmc.Material(name="Helium gap")
91          helium.set_density('g/cm3', 0.0015981)
92          helium.add_nuclide('He4', 2.4044e-04)
93
94          # Borated water coolant
95          water = openmc.Material(name="Coolant")
96          water.set_density('g/cm3', 0.740582068)
97          water.add_nuclide('H1', 4.9456e-02)
98          water.add_nuclide('O16', 2.4673e-02)
99          water.add_nuclide('B10', 7.9714e-06)
100         water.add_s_alpha_beta('c_H_in_H2O')
101
102         # Reflector water
103         refl_water = openmc.Material(name="Reflector")
104         refl_water.set_density('g/cm3', 0.740582068)
105         refl_water.add_nuclide('H1', 4.9456e-02)
106         refl_water.add_nuclide('O16', 2.4673e-02)
107         refl_water.add_s_alpha_beta('c_H_in_H2O')
108
109         # Set temperatures
110         for fuel in [fuel_1, fuel_2, fuel_3]:
111         fuel.temperature = 900
112         zircaloy.temperature = 600
113         helium.temperature = 600
114         water.temperature = 600
```

```
refl_water.temperature = 600

mats = openmc.Materials([fuel_1, fuel_2, fuel_3, zircaloy, helium,
    water, refl_water])


# GEOMETRY PARAMETERS (BEAVRS 2.0.2 dimensions)


IN = 2.54
assy_pitch = 8.466 * IN         # 21.50364 cm
pin_pitch = 0.496 * IN          # 1.25984 cm
n_pins = 17

# Fuel pin radii
fuel_r = (0.3088 / 2) * IN              # 0.39218 cm
gap_r = (0.3600 / 2 - 0.0225) * IN  # 0.40005 cm
clad_r_out = (0.3600 / 2) * IN      # 0.45720 cm

# Guide tube radii
gt_r_in = (0.442 / 2) * IN      # 0.56134 cm
gt_r_out = (0.474 / 2) * IN     # 0.60198 cm

# Core dimensions
refl_thick = 30.0               # cm
z_half = 100.0                  # cm

half_core_x = assy_pitch * nx / 2.0
half_core_y = assy_pitch * ny / 2.0
half_refl_x = half_core_x + refl_thick
half_refl_y = half_core_y + refl_thick

# Guide thimble and instrument tube positions
GT_POSITIONS = {
        (2, 2), (2, 5), (2, 8), (2, 11), (2, 14),
        (5, 2), (5, 5), (5, 8), (5, 11), (5, 14),
        (8, 2), (8, 5), (8, 11), (8, 14),
        (11, 2), (11, 5), (11, 8), (11, 11), (11, 14),
        (14, 2), (14, 5), (14, 8), (14, 11), (14, 14),
}
IT_POSITION = (8, 8)


# UNIVERSE DEFINITIONS

water_univ = openmc.Universe(cells=[openmc.Cell(fill=water)])
```

```
160
161         def make_assembly_univ(fuel_mat):
162         """Build 17x17 pin-lattice universe for a given fuel material."""
163
164         # Fuel pin cells
165         pellet_surf = openmc.ZCylinder(r=fuel_r)
166         gap_surf = openmc.ZCylinder(r=gap_r)
167         clad_surf = openmc.ZCylinder(r=clad_r_out)
168
169         fuel_pin_univ = openmc.Universe()
170         fuel_pin_univ.add_cells([
171         openmc.Cell(fill=fuel_mat, region=-pellet_surf),
172         openmc.Cell(fill=helium, region=+pellet_surf & -gap_surf),
173         openmc.Cell(fill=zircaloy, region=+gap_surf & -clad_surf),
174         openmc.Cell(fill=water, region=+clad_surf),
175         ])
176
177         # Guide thimble cells
178         gt_inner_surf = openmc.ZCylinder(r=gt_r_in)
179         gt_outer_surf = openmc.ZCylinder(r=gt_r_out)
180
181         gt_univ = openmc.Universe()
182         gt_univ.add_cells([
183         openmc.Cell(fill=water, region=-gt_inner_surf),
184         openmc.Cell(fill=zircaloy, region=+gt_inner_surf & -gt_outer_surf),
185         openmc.Cell(fill=water, region=+gt_outer_surf),
186         ])
187
188         # Pin lattice
189         pin_lat = openmc.RectLattice()
190         pin_lat.lower_left = (-pin_pitch * n_pins / 2.0, -pin_pitch *
               n_pins / 2.0)
191         pin_lat.pitch = (pin_pitch, pin_pitch)
192
193         universes = np.empty((n_pins, n_pins), dtype=object)
194         for i in range(n_pins):
195         for j in range(n_pins):
196         if (i, j) == IT_POSITION:
197         universes[i, j] = gt_univ  # Instrument tube uses GT geometry
198         elif (i, j) in GT_POSITIONS:
199         universes[i, j] = gt_univ
200         else:
201         universes[i, j] = fuel_pin_univ
202
203         pin_lat.universes = universes
204         pin_lat.outer = water_univ
```

```
    return openmc.Universe(cells=[openmc.Cell(fill=pin_lat)])

    type_to_univ = {
            1: make_assembly_univ(fuel_3),   # fresh
            2: make_assembly_univ(fuel_2),   # once-burnt
            3: make_assembly_univ(fuel_1),   # twice-burnt
    }


    # CORE LATTICE


    core_lat = openmc.RectLattice()
    core_lat.lower_left = (-half_core_x, -half_core_y)
    core_lat.pitch = (assy_pitch, assy_pitch)

    core_universes = np.empty((nx, ny), dtype=object)
    for row in range(nx):
    for col in range(ny):
    ftype = grid[row, col]
    core_universes[row, col] = (
    type_to_univ[ftype] if ftype in (1, 2, 3) else water_univ
    )

    core_lat.universes = core_universes
    core_lat.outer = water_univ


    # ROOT GEOMETRY


    z0 = openmc.ZPlane(z0=-z_half, boundary_type='reflective')
    z1 = openmc.ZPlane(z0=z_half, boundary_type='reflective')

    cx0 = openmc.XPlane(x0=-half_core_x)
    cx1 = openmc.XPlane(x0=half_core_x)
    cy0 = openmc.YPlane(y0=-half_core_y)
    cy1 = openmc.YPlane(y0=half_core_y)

    rx0 = openmc.XPlane(x0=-half_refl_x, boundary_type='vacuum')
    rx1 = openmc.XPlane(x0=half_refl_x, boundary_type='vacuum')
    ry0 = openmc.YPlane(y0=-half_refl_y, boundary_type='vacuum')
    ry1 = openmc.YPlane(y0=half_refl_y, boundary_type='vacuum')

    core_region = +cx0 & -cx1 & +cy0 & -cy1 & +z0 & -z1
```

```
    refl_region = +rx0 & -rx1 & +ry0 & -ry1 & +z0 & -z1 & ~core_region

    root_univ = openmc.Universe(cells=[
    openmc.Cell(fill=core_lat, region=core_region),
    openmc.Cell(fill=refl_water, region=refl_region),
    ])
    geom = openmc.Geometry(root_univ)


    # SETTINGS AND TALLIES


    sets = openmc.Settings()
    sets.batches = 150
    sets.inactive = 15
    sets.particles = 15000
    sets.run_mode = 'eigenvalue'

    source = openmc.IndependentSource(
    space=openmc.stats.Box(
    (-half_core_x, -half_core_y, -z_half),
    (half_core_x, half_core_y, z_half),
    )
    )
    source.constraints = {'fissionable': True}
    sets.source = [source]

    # Pin-level mesh tally (255x255 = 17x15)
    mesh = openmc.RegularMesh()
    mesh.dimension = [255, 255, 1]
    mesh.lower_left = [-half_core_x, -half_core_y, -z_half]
    mesh.upper_right = [half_core_x, half_core_y, z_half]

    power_tally = openmc.Tally(name="power")
    power_tally.filters = [openmc.MeshFilter(mesh)]
    power_tally.scores = ['fission-q-recoverable']

    tallies = openmc.Tallies([power_tally])

    return openmc.Model(geom, mats, sets, tallies)
```

Listing A.1: OpenMC 2D PWR core model with pin-level resolution

A.2 QUBO Formulation for Fuel Loading Optimization

The following code implements the QUBO Hamiltonian construction for the fuel loading optimization problem described in Chapter 3. It encodes the five constraint terms (one-hot encoding, global fuel counts, zone placement, adjacency prohibitions, and optional symmetry) as quadratic penalty terms.

```
        import pyqubo
        import numpy as np

        class NuclearCoreQUBO:
        """Construct QUBO Hamiltonian for fuel loading optimization."""

        def __init__(self, N, N_fresh, N_once, N_twice,
        peripheral, inner, adjacency, pos,
        adjacent_to_peripheral=None,
        symmetry_quadrant=None):
        """
        Parameters
        ----------
        N : int
        Number of fuel assembly positions
        N_fresh, N_once, N_twice : int
        Target fuel counts
        peripheral : list
        Indices of peripheral zone positions
        inner : list
        Indices of inner zone positions
        adjacency : dict
        Adjacency mapping: index -> list of neighbor indices
        pos : dict
        Position mapping: index -> (x, y) coordinates
        symmetry_quadrant : list, optional
        Indices in one quadrant for symmetry enforcement
        """
        self.N = N
        self.N_fresh = N_fresh
        self.N_once = N_once
        self.N_twice = N_twice
        self.B = peripheral
        self.I = inner
        self.adj = adjacency
        self.pos = pos
        self.Q1 = symmetry_quadrant

        # Compute core center for symmetry operations
```

```
        xs = [p[0] for p in self.pos.values()]
        ys = [p[1] for p in self.pos.values()]
        self.cx = (max(xs) + min(xs)) / 2.0
        self.cy = (max(ys) + min(ys)) / 2.0

        # Binary variables: x_{i,b} where b=0(fresh),1(once),2(twice)
        self.vars = {
                f'x_{i}_{b}': pyqubo.Binary(f'x_{i}_{b}')
                for i in range(N) for b in range(3)
        }

        def _get_symmetric_indices(self, i):
        """Get symmetric positions for index i under 4-fold rotational
            symmetry."""
        x, y = self.pos[i]
        rx, ry = x - self.cx, y - self.cy

        # Rotations: 90°, 180°, 270° and reflections
        targets = [
        (-ry + self.cx, rx + self.cy),      # 90° rotation
        (-rx + self.cx, -ry + self.cy),     # 180° rotation
        (ry + self.cx, -rx + self.cy),      # 270° rotation
        (-rx + self.cx, y),                 # horizontal reflection
        (x, -ry + self.cy),                 # vertical reflection
        ]

        indices = []
        for tx, ty in targets:
        key = (round(tx), round(ty))
        if key in self.coord_to_idx and self.coord_to_idx[key] != i:
        indices.append(self.coord_to_idx[key])
        return list(set(indices))

        def build_qubo(self, w_one_per_cell=1.0, w_counts=1.0,
        w_forbidden=1.0, w_adj=1.0, w_sym=1.0):
        """
        Construct the full QUBO Hamiltonian.

        Returns
        -------
        tuple
        (qubo_dict, offset) where qubo_dict maps (i,j) to coefficient
        """
        H = 0

        # H1: One fuel type per assembly
```

```
        for i in range(self.N):
        sum_fuels = sum(self.vars[f'x_{i}_{b}'] for b in range(3))
        H += w_one_per_cell * (sum_fuels - 1) ** 2

        # H2: Global fuel count constraints
        targets = [self.N_fresh, self.N_once, self.N_twice]
        for b, target in enumerate(targets):
        sum_type = sum(self.vars[f'x_{i}_{b}'] for i in range(self.N))
        H += w_counts * (sum_type - target) ** 2

        # H3: Zone placement constraints
        sum_twice_in_B = sum(self.vars[f'x_{i}_2'] for i in self.B)
        sum_fresh_in_I = sum(self.vars[f'x_{i}_0'] for i in self.I)
        H += w_forbidden * (sum_twice_in_B ** 2 + sum_fresh_in_I ** 2)

        # H4: Adjacency constraints
        for i in range(self.N):
        for j in self.adj.get(i, []):
        if j > i:  # Avoid double-counting
        # Fresh-fresh forbidden in interior
        if i not in self.B and j not in self.B:
        H += w_adj * self.vars[f'x_{i}_0'] * self.vars[f'x_{j}_0']
        # Twice-twice forbidden everywhere
        H += w_adj * self.vars[f'x_{i}_2'] * self.vars[f'x_{j}_2']

        # H5: Symmetry constraint (optional)
        if self.Q1 is not None:
        Q1_set = set(self.Q1)
        for i in self.Q1:
        for k in self._get_symmetric_indices(i):
        if k not in Q1_set:
        for b in range(3):
        diff = self.vars[f'x_{i}_{b}'] - self.vars[f'x_{k}_{b}']
        H += w_sym * (diff ** 2)

        return H.compile().to_qubo()
```

Listing A.2: QUBO formulation for nuclear core fuel loading

A.3 Feasibility Verification

The following function verifies whether a candidate loading pattern satisfies all six hard constraints defined in Chapter 3, including one-hot encoding, global fuel counts, zone placement rules, and adjacency prohibitions.

```python
import numpy as np

def check_feasibility(sample, N, B, I, adj, adjacent_to_peripheral,
N_fresh, N_once, N_twice):
"""
Verify that a loading pattern satisfies all hard constraints.

Parameters
----------
sample : dict
QUBO solution dictionary mapping variable names to binary values
N : int
Number of fuel assembly positions
B : list
Peripheral zone indices
I : list
Inner zone indices
adj : dict
Adjacency mapping
N_fresh, N_once, N_twice : int
Target fuel counts

Returns
-------
bool
True if all constraints are satisfied, False otherwise
"""
B_set = set(B)
I_set = set(I)

# Initialize fuel type arrays
fresh = np.zeros(N, dtype=int)
once = np.zeros(N, dtype=int)
twice = np.zeros(N, dtype=int)

# Parse QUBO solution
for var, val in sample.items():
if val == 1:
parts = var.split('_')
i = int(parts[1])
b = int(parts[2])
if b == 0:
fresh[i] = 1
elif b == 1:
once[i] = 1
else:
```

```
        twice[i] = 1

        # Constraint 1: exactly one fuel type per cell
        for i in range(N):
        if fresh[i] + once[i] + twice[i] != 1:
        return False

        # Constraint 2: exact global fuel counts
        if np.sum(fresh) != N_fresh:
        return False
        if np.sum(once) != N_once:
        return False
        if np.sum(twice) != N_twice:
        return False

        # Constraint 3: no twice-burnt in peripheral zone
        if any(twice[i] for i in B_set):
        return False

        # Constraint 4: no fresh in inner zone
        if any(fresh[i] for i in I_set):
        return False

        # Constraint 5: no fresh-fresh adjacency in interior
        for i in range(N):
        for j in adj.get(i, []):
        if j > i and fresh[i] and fresh[j]:
        if not (i in B_set or j in B_set):
        return False

        # Constraint 6: no twice-twice adjacency anywhere
        for i in range(N):
        for j in adj.get(i, []):
        if j > i and twice[i] and twice[j]:
        return False

        return True
```

Listing A.3: Feasibility verification for loading patterns

Plagiarism Report

A plagiarism report from iThenticate is attached to the physical copy of this thesis.